

PROJECT REPORT

On

Online job portal



I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY KAPURTHALA

In partial fulfillment of the requirement for award of
degree of Bachelor of Computer Applications (BCA)

Submitted by:

Name of the Student:
Naincy virk

University Roll no
2308730

Supervisor

Rohan

Project Guide



**Department of Computer Applications
Chandigarh College of Technology**

Landran, Mohali, (Punjab 140307)

Phone: 0172-984200, 3984206

CERTIFICATE BY COMPANY

  **Future Finders®**
Finding Future For You...

Ref. No.:- FF/6W/833

**CERTIFICATE
OF TRAINING**
PROUDLY PRESENTED TO

NAINCY VIRK

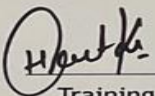
S/o D/o W/o Sh. ANRITPAL SINGH

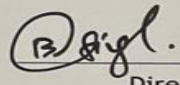
of CQC LANDRAN has Successfully

Completed his/her training in DATA ANALYST

from JUNE - 2025 to JULY - 2025 during the tenure

of the above training, we have found him/her a hardworking.


Training Co-ordinator


Director

 **MSME**
Ministry of MSME, Govt. of India
Reg. no. - CH010009429

 **ISO**
International Organization for Standardization
9001:2015

 **IAF**
MEMBER OF MULTILATERAL
RECOGNITION ARRANGEMENT

 **eiaci**
عمرات العالمية للاعتمادات
Emirates International Accreditation Centre
CB-QMS-107

Grade - A ☐ A+ ☒ B ☐ B+ ☐ C ☐

  info@futurefinders.in
 www.futurefinders.in
 Plot No. F - 465, K&B Tower, 2nd
Floor, Phase 8B, Sector 75, Mohali.
 Mobile 9855 908 009

FACULTY DECLARATION

I hereby declare that the student Ms. Naincy virk_of BCA has undergone her Project under my periodic guidance on the Project titled “(Online job portal)”.

Further I hereby declare that the student was periodically in touch with me during her training period and the work done by student is genuine & original.

(Signature of Supervisor)

CANDIDATE DECLARATION

I hereby declare that the Project Report entitled ("**Online Job Portal**") is an authentic record of my own work as requirements of 5th semester academic during the period from **9th June 2025** to **23th July 2025** for the award of degree of BCA (Bachelor of computer Applications, Chandigarh College of Technology- CGC, Landran , Mohali under the guidance of (Mr. Rohan).

(Signature of student)

(Naincy virk)
(2308730)

Date: 24-07-2025

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Signatures

Examined by:

1.

2.

3.

4.

**Head of Department
(Signature and Seal)**

ACKNOWLEDGMENT

I take this opportunity to express my sincere gratitude to the Director- Principal **Dr. Palki Saheb Kaur** Chandigarh College of Technology, Landran for providing this opportunity to carry out the present work.

I am highly grateful to the **Dr. Inderpreet Kaur** HOD CCT, Chandigarh College Of Technology, Landran (Mohali). I would like to express my gratitude to other faculty members of Computer Applications department of CCT, Landran for providing academic inputs, guidance & Encouragement throughout the training period. The help rendered by **Mr. Rohan**, Supervisor for Experimentation is greatly acknowledged. I would also like to acknowledge the help of my Project Partner, **Mrinal**, who helped me all along the building of this project. Finally, I express my indebtedness to all who have directly or indirectly contributed to the successful completion of my semester training.

ABOUT COMPANY

As a final semester student pursuing a degree in Computer Applications, I had the opportunity to gain valuable industry experience through a 45 days Training program with **Solitaire Infosys Pvt. Ltd.**

During my training, I received a certification in Java Full Stack , which provided me with a strong foundation in programming and software development. However, my major project revolved around the development of an online job portal using Java Full Stack. This project required extensive work on my part, collaborating with my project partner, to meet the requirements for our Fifth Semester Project. Our focus was on implementing various features such as user authentication, job listings, resume uploads, and search functionalities to create a robust and user-friendly platform for job seekers and employers. The project demanded independent effort outside of the Institute to ensure its successful completion.

Solitaire Infosys is a leading Software and Web Application Development Company, based in Mohali (Chandigarh), that provides high-quality comprehensive services to enterprises across a wide range of platforms and technologies. Our major areas of expertise are in providing quality, cost-effective software or web development services. Solitaire Infosys Pvt. Ltd. is an acclaimed IT service provider contributing its part in the development of many businesses around the globe. We socialize with our clients to get a superior cognizance of their business and requirements and help them in fabricating websites and applications for their business.

Services Offered: IT Consulting , Software Design & Prototyping Custom Software Development, Software Testing & QA Services ,Web Application Development, Application Support & Software Maintenance ,Web Designing Solutions (Graphics, Brand & Logo Designing) , Web Development Solutions (PHP, .Net, HTML, C++, Java) , Mobile Application Development (Android, Ios) , Internet Marketing (SEO, SEM, SMM).

ABSTRACT

Job portal service was developed for creating an interactive job vacancy form for candidates. This web application manage updates both from the job seekers as well as the companies. It's unique development methodology helps in acquiring the client and candidate information and separating them according to the job requirements and vacancies.

The online access to it provides details of the job. An employer being registered in the web site has the facility to use the services. Being an authorized user can publish vacancy details and can search no of Employees on portal and also he can search candidates on basis of the key skill which employee provides on registration.

The project has been planned to be having the view of distributed architecture, with centralized storage of the database. The application for the storage of the data has been planned. Using the constructs of MS-SQL Server and all the user interfaces have been designed using the JSP technologies. The database connectivity is planned using the "SQL Connection" methodology. The standards of security and data protective mechanism have been given a big choice for proper usage. The application takes care of different modules and their associated reports, which are produced as per the applicable strategies and standards that are put forwarded by the administrative staff .

Table of Contents

CONTENT	PAGE NO.
Certificate	i
Candidate Declaration	ii
Acknowledgement.....	iii
About Company	iv
Abstract	v
Table of Contents	vi
List of Figures	viii
List of Tables.....	ix
Chapter 1 : Introduction	1
1.1 Brief Overview of Work.....	1
1.2 Objective	1
1.3 Scope	1
1.4 Project Modules	2
1.4.1 Registration.....	2
1.4.2 Search	2
1.4.3 Job Post.....	2
1.4.4 Manage Account.....	2
1.5 Project Requirements	3
1.5.1 Hardware	3
1.5.2 Software.....	3
Chapter 2 : System Analysis	4
2.1 Literature Review	4
2.2 Project Feasibility Study.....	5
2.2.1 Technical Feasibility	5

2.2.2 Economical Feasibility	5
2.2.3 Operational Feasibility	5
2.4 Project Timeline Chart	6
2.5 Detailed Module Description with all Functionalities	7
2.5.1 Registration	7
2.5.2 Job Post	7
2.5.3 Search	7
2.5.4 Manage Account	7
Chapter 3 : System Design	8
3.1 Use Case Diagrams	8
3.2 Data Flow Diagrams.....	9
3.3 Class Diagram	11
3.4 Entity Relationship Diagram	12
3.5 Activity Diagram	12
3.6 Data Dictionary	15
Chapter 4 : Software Tools.....	25
4.1 Overview of Java Technology	25
4.1.1 History of Java	25
4.1.2 Features of Java.....	25
4.1.3 Java and Internet.....	26
4.1.4 Java And World Wide Web.....	26
4.1.5 Java Environment	26
4.1.6 Java Architecture	27
4.1.7 Java Virtual Machine	27
4.18 Java Paradigm	27
4.2 About HTML	27

4.3 Java Script	28
4.4 Introduction to Servlets	29
4.5 Introduction to MySQL	49
Chapter 5 : Implementation and Testing	50
5.1 User Interface and snapshots	53
5.2 Test Cases and Result.....	62
Chapter 6 : Conclusion & Future work	62
References	64

List Of Figures

CONTENT	PAGE NO.
Figure 2.1- Timeline chart	6
Figure 3.1- Use case Diagram	8
Figure 3.2- DFD 0 level Diagram.....	9
Figure 3.3- DFD 1 level Diagram.....	9
Figure 3.4- DFD 2 level Diagram.....	10
Figure 3.5- Class Diagram.....	11
Figure 3.6- ER Diagram	12
Figure 3.7- Company Activity Diagram.....	12
Figure 3.8- User Activity Diagram.....	13
Figure 3.9- Admin Activity Diagram	14
Figure 5.1- Home page I.....	53
Figure 5.2- Home page II	53
Figure 5.3- Company Registration Page.....	54
Figure 5.4- Company Login Form.....	54
Figure 5.5- Company Job Posting	55
Figure 5.6- User Registration Form.....	55
Figure 5.7- User Login Form	56

List of Tables

CONTENT	PAGE NO.
Table 1- employer table.....	15
Table 2 – user table	16
Table 3- job_category table	17
Table 4- job table.....	18
Table 5- job_applications table	19
Table 6- like_log table.....	20
Table 7- comments table	21
Table 8- watchlist table	22
Table 9- message table	23
Table 10- feedback table	24
Table 11- city table.....	24
Table 12- Test Case.....	57

Chapter 1 : Introduction

1.1 Brief Overview of Work

Now a day, we know that searching of jobs is so difficult in proficient areas the portal developed for the providing the simple and good job searching. With the help of this portal easily the job seeker can submit their resume and get the lot of opportunity of the job related to their profile. And by this website the companies or employer can also find the good and well profiled resume.

1.2 Objective

The online job Portal System that is to be developed provides the members with jobs information, online applying for jobs and many other facilities. This system provides service to the job applicants to search for working opportunities.

Job Portal will allow job provider to establish one to one relationships with candidates. This Portal will primarily focus on the posting and management of job vacancies. This system is designed such that ultimately all vacancies will be posted online and would offer employers the facilities to post their vacancies online. It helps to review and manage the resulting applications efficiently through the web. Employer can also find the resume according to key skill in very less amount of time.

1.3 Scope

As of Indian market, there is ample opportunities for the job portal sites, as more and more number of educated and skilled young people are coming out each and every year. Also, as the growth rate of India is zooming to be at a healthy rate over 7%, so it is boom time for corporate also. So, more and more number of lucrative careers will be available for the job seekers. So, it is now the right period for the job portal sites to think out of the box, and to make most of the opportunities available.

1.4 Project Modules

1.4.1 Registration

Employee or Employer can register with valid details like contact details, experience details, profile details.

1.4.2 Search

Employee Can Search job according to their interest. And also apply for that job. Employer search candidates for their requirements using keyword. Employer also can communicate with employee for their any other query or information via send message.

1.4.3 Job Post

Employer post job for their organization. And include job vacancy, salary details, working hours, designation details, experienced details.

1.4.4 Manage Account

Employee can also delete his/her account anytime.

Admin Can Manage Employee and Employer Details. Admin observed Users Action like job posting, candidate details false or not.

1.5 Project Requirements

1.5.1 Hardware

The system requires the following hardware:

- RAM: 1 GB (further increase that as per requirement.)
- Hard Disk: 80 GB (further increase that as per requirement.)
- Display: 1024 * 768, True Type Color-32 Bit
- Mouse: Any Normal Mouse.
- Keyboard: Any window Supported Keyboard.

1.5.2 Software

- Database Server : Microsoft SQL Server
- Web Server : Internet Information Server
- Technologies : HTML,CSS,ASP.NET,JavaScript,JQuery

Chapter 2 : System Analysis

2.1 Literature Review

Job Procurement: Old and New Ways Job seeking usually involves different ways to look for jobs such as through personal contacts, direct telephone calls to employers, job agency office, scanning online job listings, etc. Before the Internet, became widely uses as a method of seeking jobs, jobseekers spent a lots of time using various methods to look for job openings. Today, jobseekers use online methods which are very convenient and save a lot of time. Galanaki lists the following methods to be the traditional (old) ways for recruitment:

- 1) Employment recruitment agencies
- 2) Job fairs
- 3) Advertising in the mass media such as newspapers
- 4) Management Consultants
- 5) Advertisement in television and radio
- 6) Existing employee contacts
- 7) Schools colleges or universities students services department
- 8) Workers or professional referrals

These old job seeking methods are too slow, stressful, challenging and also lack quality. In addition, the applicants have to consider the cost and the amount of time to get the information they need, and other preparations they have to make. Finding all available job vacancies is a main step at in the job-seeking process. The Internet is now a powerful tool that jobseekers can use. Today, there are many sites that advertise job positions to be filled by people with certain skills in various fields. The Internet plays an important role in the area of human resource planning and development. Most planning and development organizations are now using computer technology and the Internet for staff recruitment. It should be noted that although the Internet has facilitated the process of job-seeking, it has not replaced the traditional methods, completely.

Importance of Job Portals

In the age of technology, the Internet has become the main source of information for jobseekers. Large corporations, Institutions, and universities include information on career Prospects on their websites. According to a survey, 70% of the workforce uses websites or portals on the Internet to

Search for jobs in France. These websites or portals provide search engine to access information on job opportunities.

2.2 Project Feasibility Study

2.2.1 Technical Feasibility

Technical feasibility study is concerned with specifying equipment and software that will successfully satisfy the user requirement; the technical needs of the system may vary considerably.

The facility to produce outputs in a given time. Our project is a web based application which is based on client-server based application. In this application every page as output is render from server to client so it is necessary that the page should be rendered in time. For this I have avoided more and more code in the page- load event.

2.2.2 Economical Feasibility

Economical feasibility is the measure to determine the cost and benefit of the proposed system. A project is economical feasible which is under the estimated cost for its development. These benefits and costs may be tangible or intangible. Job Portal is the cost-effective project in which there is less possibility of intangible cost so there is no difficulty to determine the cost of the project.

2.2.3 Operational Feasibility

Operation feasibility is used to check whether the project is operationally feasible or not. Our project is mainly different from the other system because of its web-support feature. So the measure for operational feasibility is something different from other system. Generally the operational feasibility is related to organization aspects.

The change determination is as such that early product were either a man or group of men or the jobs based manual but now a day with the advent of Internet technology.

2.4 Project Timeline Chart

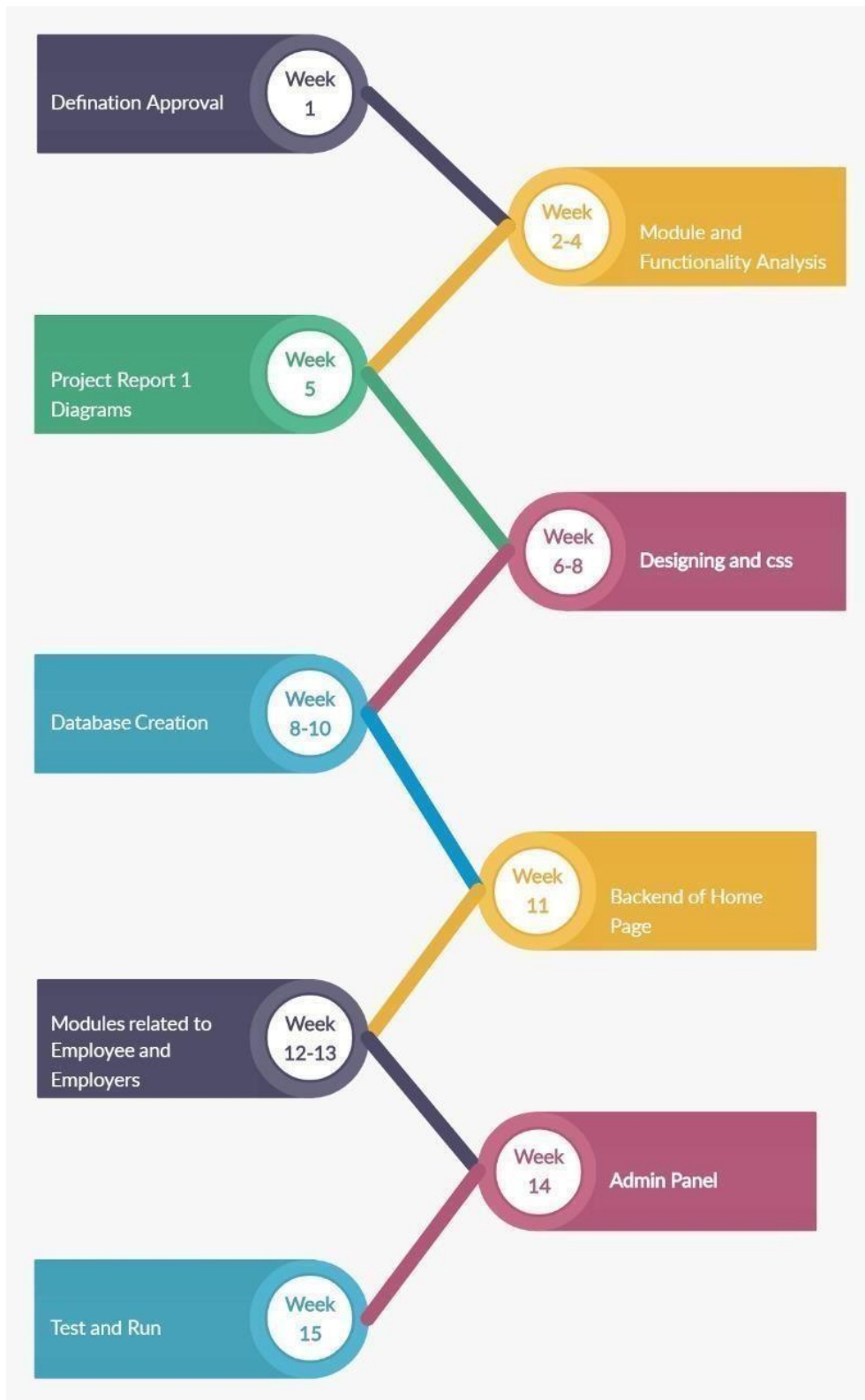


Figure 2.1- Timeline chart

2.5 Detailed Module Description with all Functionalities

2.5.1 Registration

In the registration module job seeker have to include all the details like personal details, contact details, education details like school , graduation, post-graduation, course certification details etc. Also job seeker has to add his experience details, job requirements and uploading resume and photo.

While job recruiter has to add his contact details and organization details for the registration and upload company logo and profile.

2.5.2 Job Post

Employer can post a job by providing all the job details like qualifications details, requirements for the job, designation details, job salary details and also provide type of jobs. They also can delete the jobs whenever they want. After successfully posted a job it will be available for all the job seekers who are searching for a job. And it will be available on home page as recently posted job.

2.5.3 Search

Employee Can Search job according to their interest. And also apply for that job or they can add into wishlist for future whenever they find for job for that company then they easily find out company from wishlist.

Employer search candidates for their requirements using keyword like technology. And also can communicate with employee for their any other query or information via send message .and also employer see the resume of applicants.

2.5.4 Manage Account

While employers can manage their job postings. And providing all the job details like qualifications details, requirements for the job, designation details, job salary details and also provide type of jobs. They also can delete the jobs whenever they want.

While employee can manage their wishlist, applied for job and also getting full details of employer. Employees can delete their account anytime. Also they can apply for the different jobs according to their interests

Chapter 3 : System Design

3.1 Use Case Diagrams

Use Case

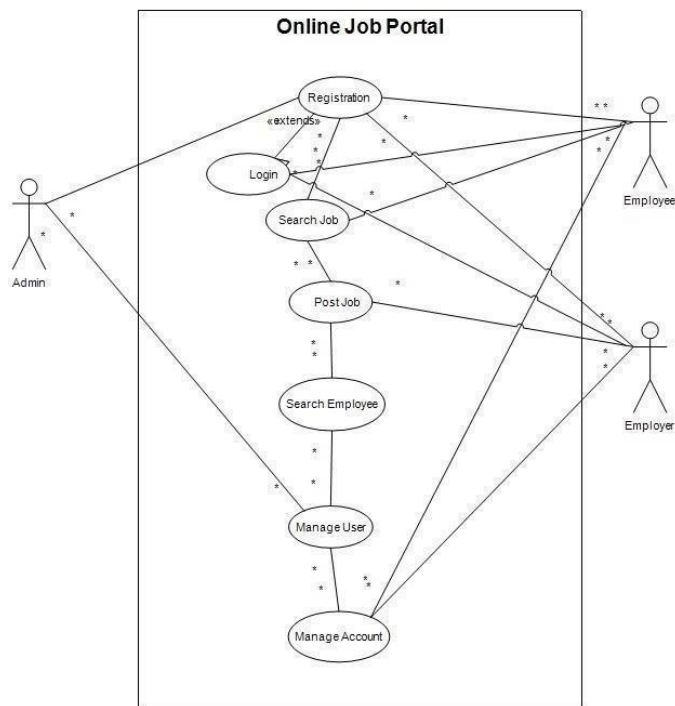


Figure 3.1- Use case Diagram

3.2 Data Flow Diagrams

3.2.1 Context-Level (Level 0) DFD

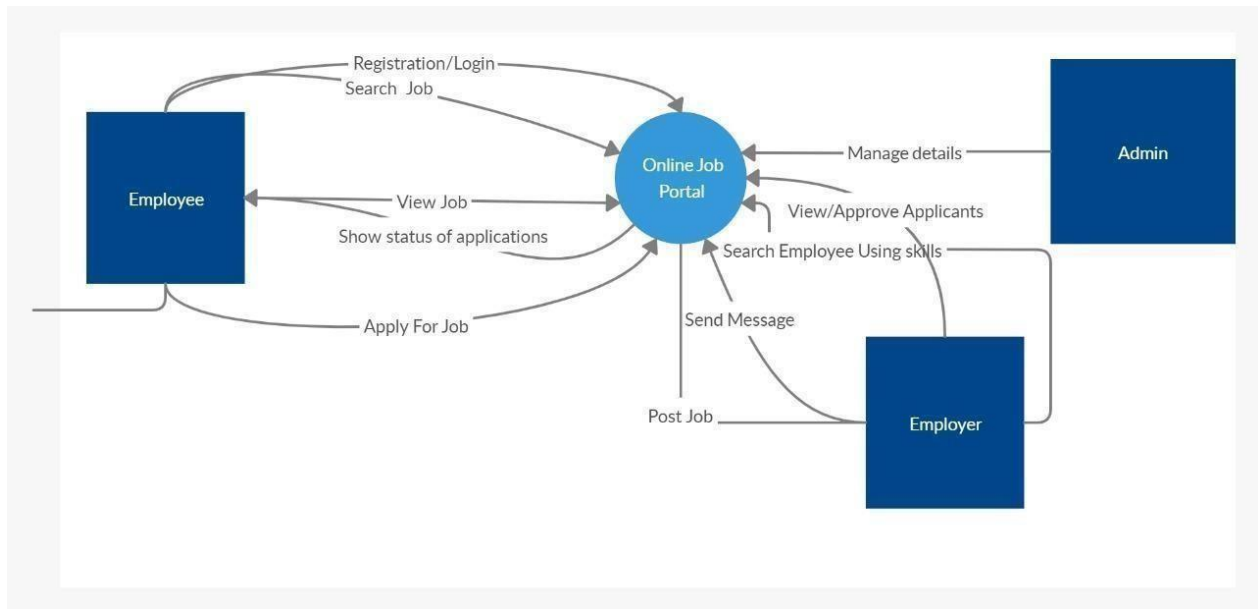


Figure 3.2- DFD 0 level Diagram

3.3.2 Level 1 DFD

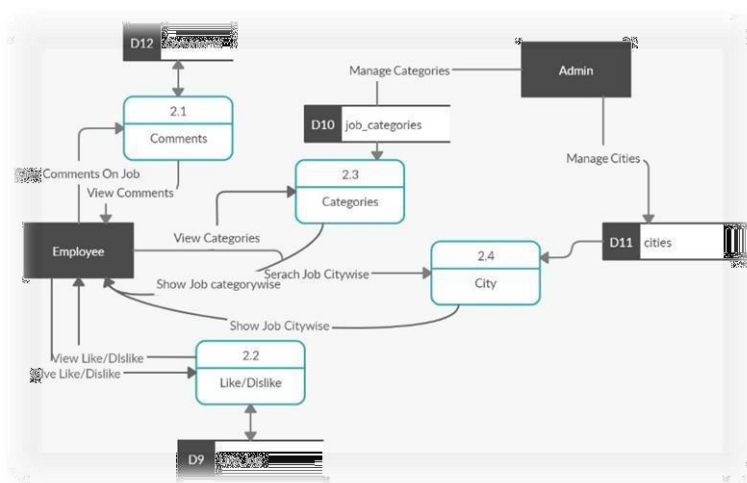


Figure 3.3- DFD 1 level Diagram

3.2.2 Level 2 DFD

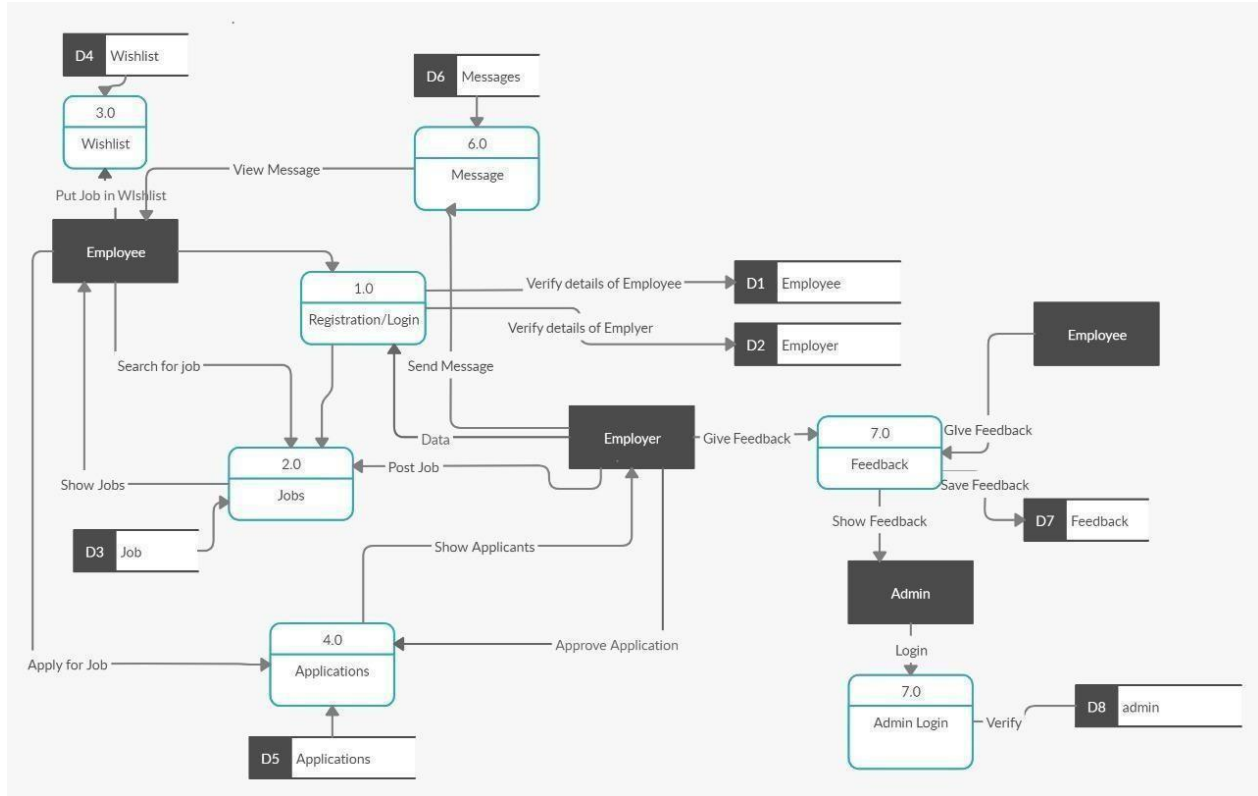


Figure 3.4- DFD 2 level Diagram

3.3 Class Diagram

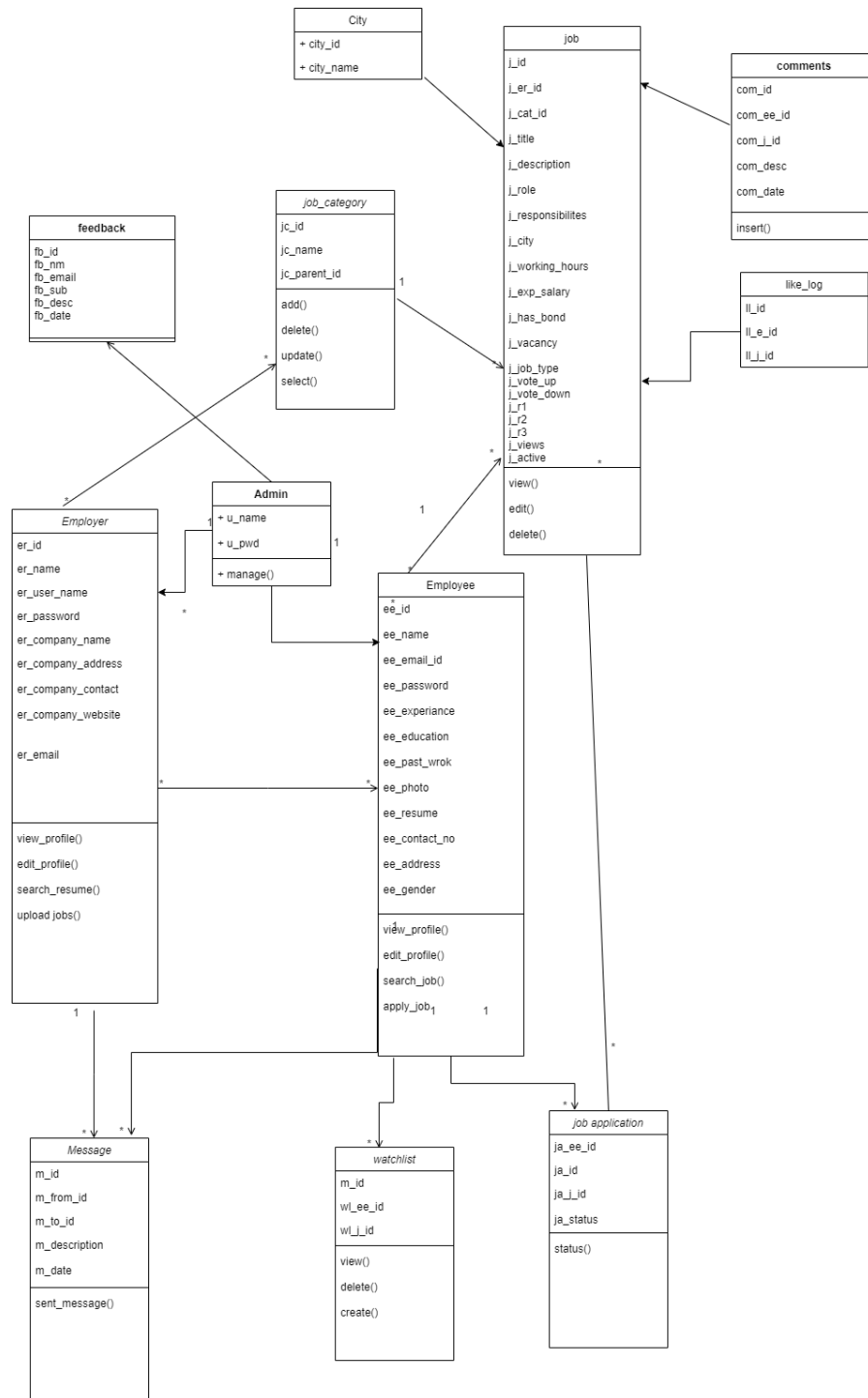


Figure 3.5- Class Diagram

3.4 Entity Relationship Diagram

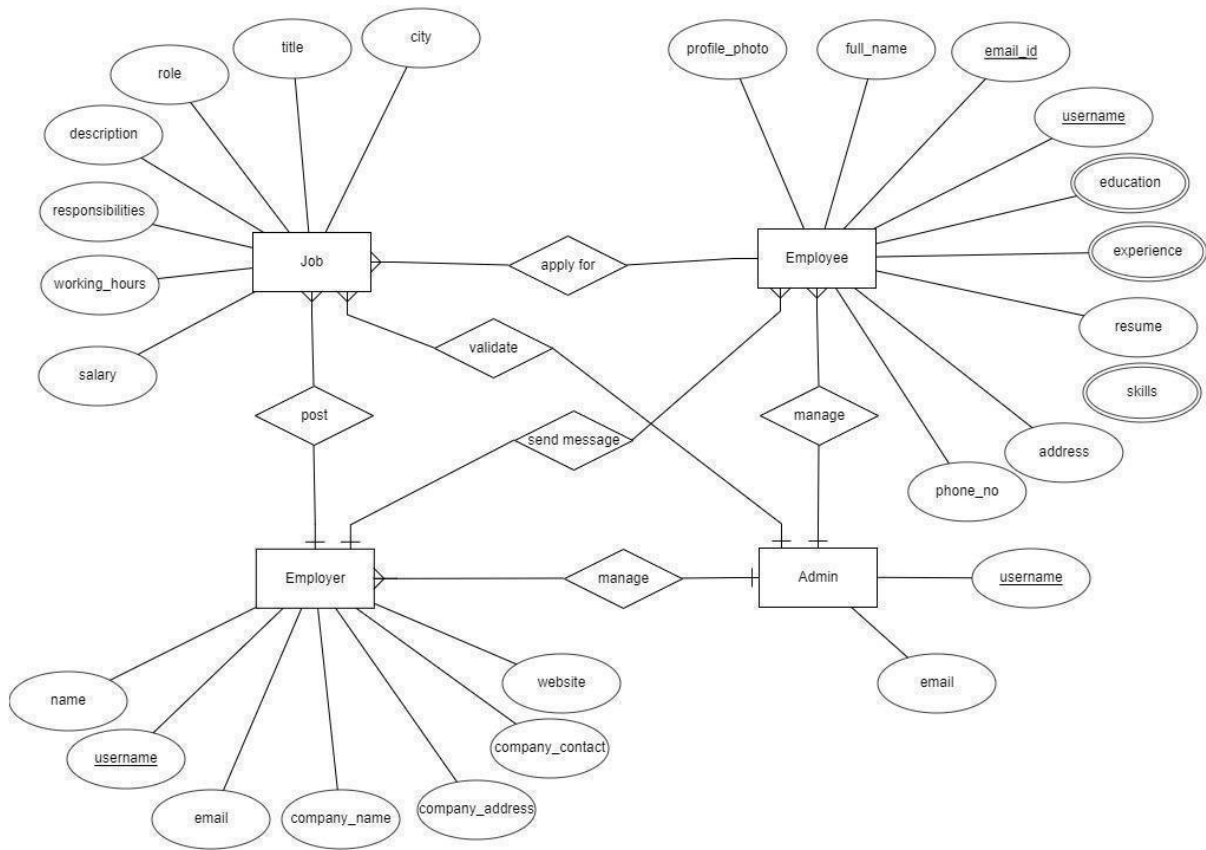


Figure 3.6- ER Diagram

3.5 Activity Diagram

3.5.1 Company

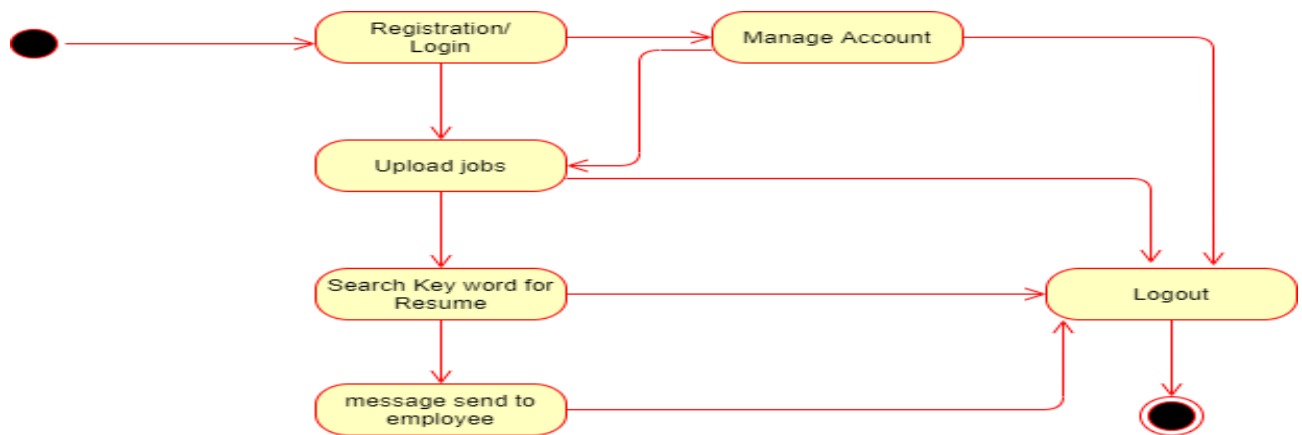


Figure 3.7- Employer Activity Diagram

3.5.2 User

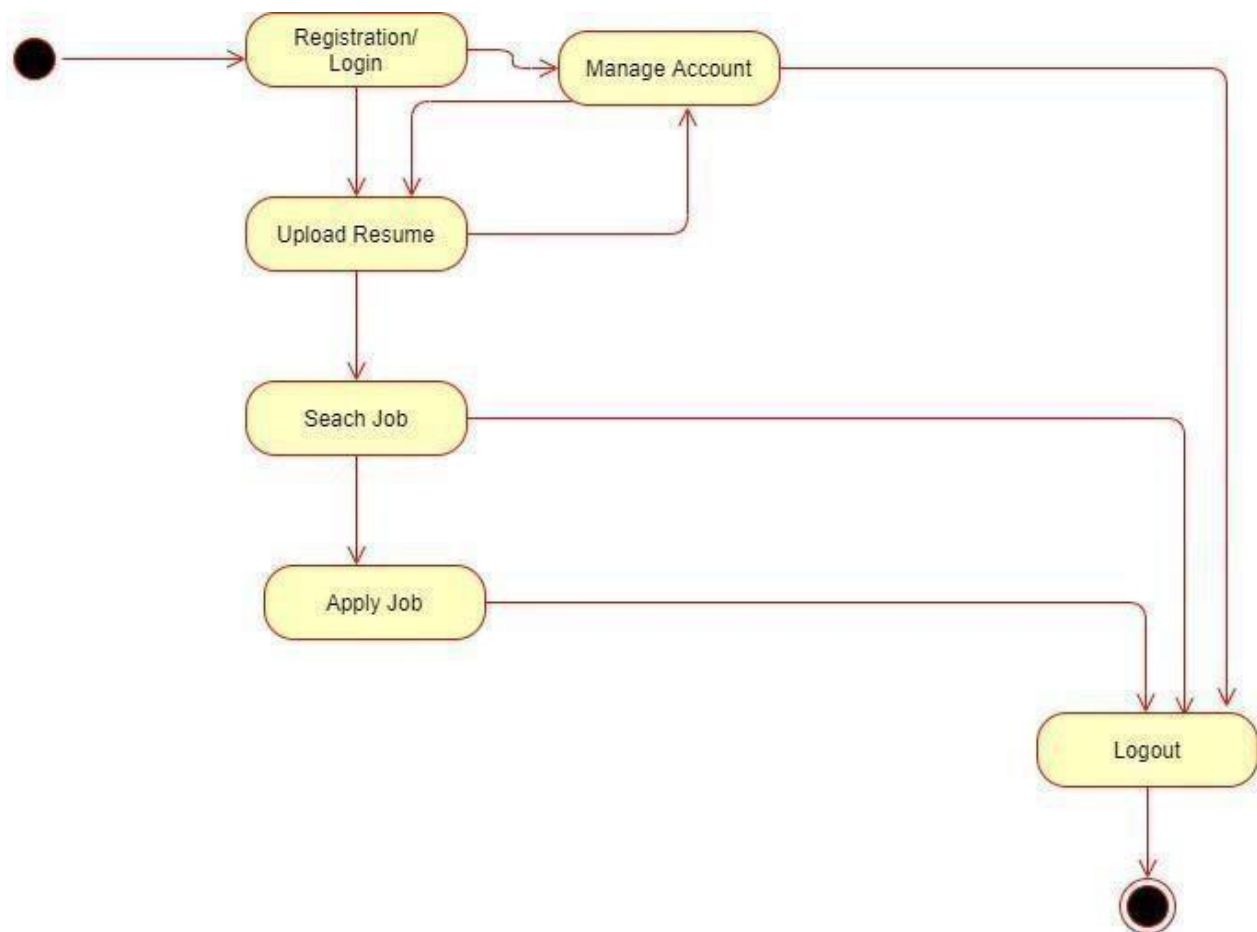


Figure 3.8- User Activity Diagram

3.5.3 Admin

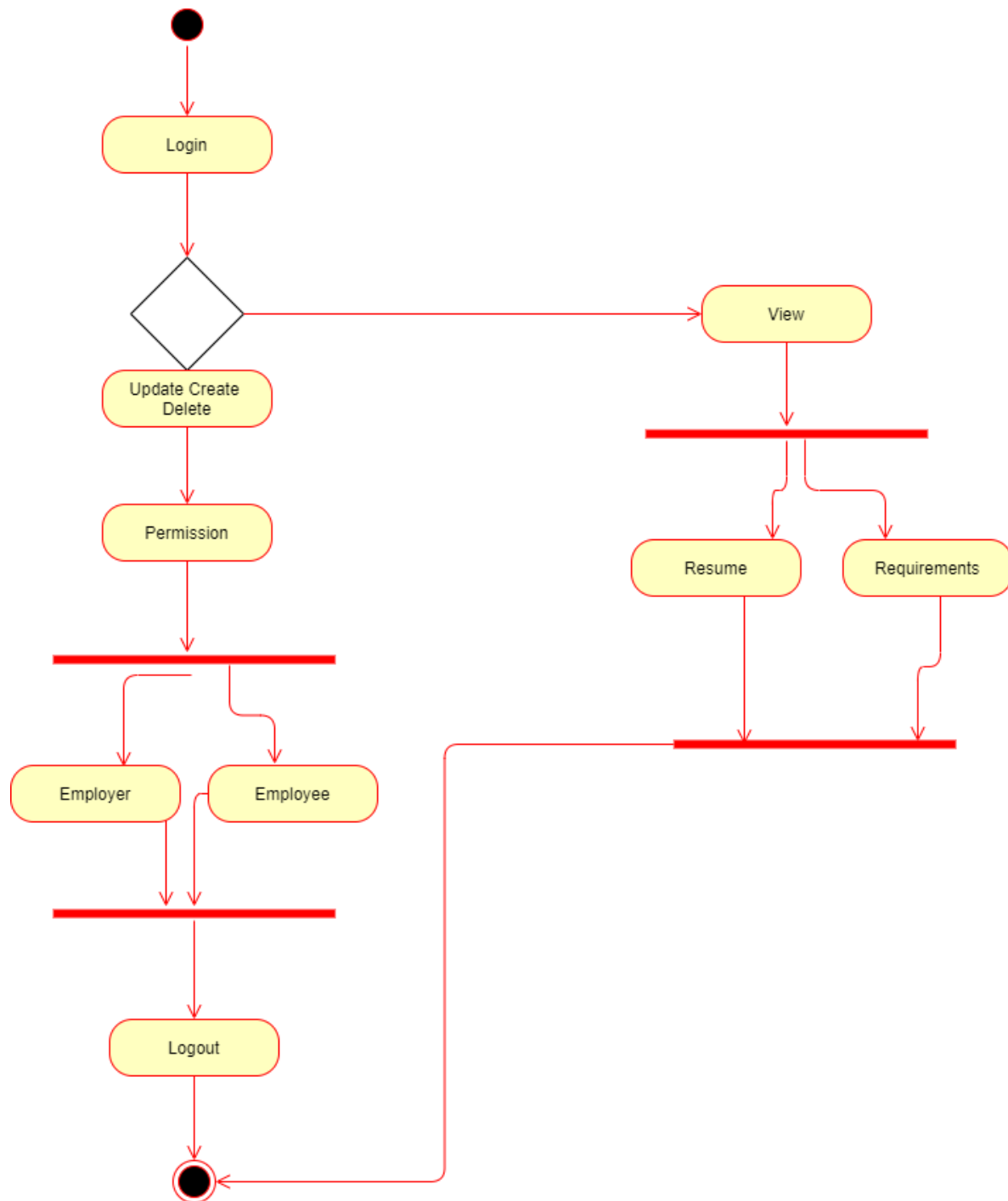


Figure 3.9- Admin Activity Diagram

3.6 Data Dictionary

Table 1- employer table

Table 1:				
Name:		employer		
Description:		Records information regarding employer.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	er_id	int	primary key	Holds unique id of employer.
2	er_full_name	varchar(50)	Null	Holds full name of the employer.
3	er_user_name	varchar(50)	Null	Holds user name of the employer.
4	er_email	varchar(50)	Null	Holds e-mail id of the employer.
5	er_pwd	varchar(50)	Null	Holds password of employer.
6	er_comp_name	varchar(50)	Null	Holds company name.
7	er_comp_address	text	Null	Holds company address.
8	er_comp_phone	varchar(20)	Null	Holds company phone number.
9	er_comp_web_add	varchar(50)	Null	Holds company website URL.
10	er_comp_fax	varchar(50)	Null	Holds company fax number.
11	er_comp_logo	varchar(max)	Null	Holds company logo.
12	er_comp_profile	varchar(max)	Null	Holds company profile
13	er_sec_que	varchar(50)	Null	Holds security question
14	er_sec_ans	varchar(50)	Null	Holds security answer

Table 2 – user table

Table 2:				
Name:		user		
Description:		Records information regarding employee.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	ee_id	int	primary key	Holds the employee-id.
2	ee_full_name	varchar(50)	Null	Holds full name of employee.
3	ee_user_name	varchar(50)	Null	Holds user name of employee.
4	ee_email	varchar(50)	Null	Holds email id of employee.
5	ee_pwd	varchar(250)	Null	Holds password of employee.
6	ee_education	varchar(50)	Null	Holds education detail of employee.
7	ee_experience	Int	Null	Holds experience detail of employee.
8	ee_past_work	Text	Null	Holds past work detail of employee.
9	ee_photo	varchar(255)	Null	Holds photograph of employee.
10	ee_resume	varchar(255)	Null	Holds resume of employee.
11	ee_phone_no	varchar(50)	Null	Holds phone no of employee.
12	ee_address	Text	Null	Holds address of employee.
13	ee_gender	Char(1)	Null	Holds gender of employee.

14	ee_sec_que	varchar(50)	Null	Holds security question of employee
15	ee_sec_ans	varchar(50)	Null	Holds security answer of employee
16	ee_first_login	int	Null	Holds status
17	ee_ks1	varchar(50)	Null	Holds key skill 1
18	ee_ks2	varchar(50)	Null	Holds key skill 2
19	ee_ks3	varchar(50)	Null	Holds key skill 3
20	ee_int_city	varchar(50)	Null	Holds interested city
21	ee_int_category	varchar(50)	Null	Holds interested category
22	ee_int_sub_category	varchar(50)	Null	Holds interested sub category

Table 3- job_category table

Table 3:				
Name:		job_category		
Description:		Record information about categories of job.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	jc_id	Int	primary key	Holds id of job category.
2	jc_name	varchar(50)	not null	Holds job category name.
3	jc_parent_id	Int	foreign key	Holds id of parent category.

Table 4- job table

Table 4:				
Name:		Job		
Description:		Records information about job posted by recruiter.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	j_id	int	primary key	Holds id of job.
2	j_er_id	int	foreign key	Holds id of employer.
3	j_jc_id	int	foreign key	Holds id of job category.
4	j_sub_cat_id	Int	Foreign key	Holds id of job sub category
5	j_title	varchar(150)	Null	Holds title of the job.
6	j_exp_required	varchar(50)	Null	Holds job experienced required
7	j_min_edu	varchar(50)	Null	Holds job minimum education
8	j_desc	Text	Null	Holds description of job.
9	j_role	Text	Null	Holds role of job.
10	j_respo	Text	Null	Holds responsibilities of job.
11	j_city	varchar(50)	Null	Holds city name of job.
12	j_work_hours	Int	Null	Holds working hours detail of job
13	j_exp_salary	varchar(50)	Null	Holds expected salary of job.
14	j_has_bond	varchar(50)	Null	Holds information of bond information.

15	j_vacancy	varchar(50)	Null	Holds information of vacancy
16	j_job_type	varchar(50)	Null	Holds information of job type
17	j_post_date	varchar(50)	Null	Holds information of job post date
18	j_vote_up	Int	Null	Holds information of job likes.
19	j_vote_down	Int	Null	Holds information of job dislikes.
20	j_views	Int	Null	Holds information of job views.
21	j_active	varchar(50)	Null	Holds information of job active or not

Table 5- job_applications table

Table 5:				
Name:		job_applications		
Description:		Records information about job application		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	ja_id	int	primary key	Holds the id of job application.
2	ja_ee_id	int	foreign key	Holds the id of employee.
3	ja_j_id	int	foreign key	Holds the information for which job applied.
4	ja_status	varchar(50)	not null	Holds the status of application of job.

Table 6- like_log table

Table 6:				
Name:		like_log		
Description:		Records information about like.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	ll_id	int	primary key	Holds id of like.
2	ll_ee_id	int	foreign key	Holds id of employee.
3	ll_j_id	int	foreign key	Holds id of job.

Table 7- comments table

Table 7:				
Name:		comments		
Description:		Records information about comment which user does.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	com_id	int	primary key	Holds qualification type id
2	com_ee_id	int	foreign key	Holds type of qualification like
3	com_j_id	int	foreign key	Holds job id.
4	com_description	Text	Null	Holds comment description.
5	com_date	varchar(50)	Null	Holds date on which commented by user.

Table 8- watchlist table

Table 8:				
Name:		watchlist		
Description:		Record information about watchlist.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	wl_id	Int	primary key	Holds unique id watch list.
2	wl_ee_id	Int	foreign key	Holds watchlist id of employee.
3	wl_j_id	Int	foreign key	Holds watchlist id of job

Table 9- message table

Table 9:				
Name:		message		
Description:		Records information about message sent by user.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	m_id	number(10)	primary key	Holds the unique id for message id.
2	m_from_id	number(10)	foreign key	Holds the id of sender.
3	m_to_id	number(10)	foreign key	Holds the id of receiver.
4	m_description	text(100)	Null	Holds the description regarding message.
5	m_date	date	Null	Holds the date of message.

Table 10- feedback table

Table 10:				
Name:		Feedback		
Description:		Record information about feedback.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	fb_id	Int	Primary Key	Holds unique id for feedback.
2	fb_nm	varchar(50)	Null	Holds the name of feedback.
3	fb_email	Varchar(50)	Null	Holds the email of sender
4	fb_sub	varchar(50)	Null	Holds the subject of feedback.
5	fb_description	Text	Null	Holds description of feedback.
6	fb_date	varchar(50)	Null	Holds date of feedback.

Table 11- city table

Table 11:				
Name:		City		
Description:		Record information about city.		
Fields				
Sr. No.	Field Name	Field Type	Constraints	Description
1	city_id	Int	Primary Key	Holds unique id for city.
2	city_name	varchar(50)	Null	Holds the name of city.

Chapter 4 : Software Tools

4.1 Overview Of Java Technology

4.1.1 History Of Java

Java language was developed by James Gosling and his team at sun micro systems and released formally in 1995. Its former name is oak. Java Development Kit

1.0 was released in 1996. to popularize java and is freely available on Internet.

Java is loosely based on C++ syntax, and is meant to be Object-Oriented Structure of java is midway between an interpreted and a compiled language . java programs are compiled by the java compiler into byte codes which are secure and portable across different platforms . These byte codes are essentially instructions encapsulated in single type, to what is known as java virtual machine (JVM) which resides in standard browser.

JVM verifies these byte codes when downloaded by the browser for integrity. Java Virtual Machines available for almost all Operating Systems. JVM converts these byte codes into machine specific instructions at runtime.

4.1.2 Features Of Java

- Java is object-oriented language and supports encapsulation, inheritance , polymorphism and dynamic binding , but does not support multiple inheritance. Everything in java is an object except some primitive datatypes .
- Java is portable architecture neutral that is java programs once compiled can be executed on any machine that is enabled.
- Java is distributed in its approach and used for internet programming.
- Java is robust, secured, high performing and dynamic in nature.
- Java supports multithreading. There for different parts of the program can be executed at the same time .

4.1.3 Java And Internet

Java is strongly associated with internet and known as internet programming language. Internet users can use java to create applet programs and run them locally using java enabled browser search as hot java. Applets can be downloaded from remote machine via internet and run it on local machine .

4.1.4 Java And World Wide Web

World wide web is an open ended information retrieval system designed to be used in the distributed environment. This system contains web pages that provide both information and controls. We can navigate to a new web page in any direction. This is made possible with HTML java was meant to be used in distributed environment such as internet. So java could be easily incorporated into the web system and is capable of supporting animation graphics , games and other special effect. The web has become more dynamic and interactive with support of java. We can run a java program on remote machine over internet with the support of web .

4.1.5 Java Environment

Java environment includes a large no. of tools which are part of the system known as java development kit (JDK) and hundreds of classes, methods, and interfaces grouped into packages forms part of java standard library (JSL).

4.1.6 Java Architecture

Java architecture provides a portable , robust , high performing environment for development. Java provides portability by compiling the byte codes for the java virtual machine which are then interpreted on each platform by the runtime environment . Java also provides stringent compile and runtime checking and automatic memorymanagement in order to ensure solid code .

4.1.7 Java Virtual Machine

When we compile the code, java compiler creates machine code (byte code) for a hypothetical machine called java virtual machine (JVM). The JVM will execute thebyte code and overcomes the issue of portability .The code is written and compile for one machine and interpreted all other machines. This machine is called java virtual machine. .

4.1.8 Paradigm Of Java

- Dynamic down loading applets(small application programs);
- Elimination of flatware phenomenon that is providing those features of a product that user needs at a time. The remaining features of a product can remain in the server.
- Changing economic model of the software
- Up-to-date software availability
- Supports network entire computing

4.2 About Html

HTML (hyper text markup language) is a language used to create hyper text documents that have hyper links embedded in them . it consists of tags embedded in the text of a document with HTML. We can build web pages or web document s. it is basically a formatting language and not a programming language. The browser reading the document interprets mark up tags to help format the document for subsequent display to a reader. HTML is a language for describing structured documents. HTML isa platform independent. WWW(world wide web) pages are written using HTML. HTML tags control in part the representation of the WWW page when view with web browser . Examples of browsers used to be web pages include:

- Netscape
- Internet Explorer

Java Script

Java script is a general purpose prototype based , object oriented scripting language developed jointly by sun and Netscape and is meant for the WWW . it is designed to be embedded in diverse applications and systems , with out consuming much memory . java script borrows most of its syntax from java but also inherits from AWK and PERL , with some indirect influence from self in its object prototype system.

Java scripts dynamically typed that is programs do not declare variable types, and the type of variable is unrestricted and can change at runtime. source can be generated at run time and evaluated against an arbitrary scope. Typical implementations compile by translating source into a specified byte code format, to check syntax and source consistency. Note that the availability to generate and interpret programs at runtime implies the presence of a compiler at runtime.

Java script is a high level scripting language that does not depend on or expose particular machine representations or operating system services. It provides automatic storage management, typically using a garbage collector.

Features:

- Java script is embedded into HTML documents and is executed with in them.
- Java script is browser dependent
- Java script is an interpreted language that can be interpreted by the browser at run time.
- Java script is loosely typed language
- Java script is an object based language.
- Java script is an Event-Driven language and supports event handlers tospecify the functionality of a button.

Advantages

- Java script can be used for client side application
- Java script provides means to contain multi frame windows forpresentation of the web.
- It creates interactive forms and client side lookup tables .
- Java script provides basic data validation before it is sent to the server. EG : login and password checking or whether the values entered are correct or whether all fields in a from are filled and reduced network traffic .

6.3 Introduction to Servlets :

Servlets provide a Java^(TM)-based solution used to address the problems currently associated with doing server-side programming, including inextensible scripting solutions, platform-specific APIs, and incomplete interfaces . Servlets are objects that conform to a specific interface that can be plugged into aJava-based server. Servlets are to the server-side what applets are to the client-side -- object byte codes that can be dynamically loaded off the net. They differ from applets in that they are faceless objects (without graphics or a GUI component). They serve as platform-independent, dynamically-loadable, pluggable helper byte code objects on the server side that can be used to dynamically extend server-side functionality.

What is a Servlets?

Servlets are modules that extend request/response-oriented servers, such as Java-enabled web servers. For example, a `SERVLET` might be responsible for taking data in an HTML order-entry form and applying the business logic used to update a company's order database servlets are to servers what applets are to browsers. Unlike applets, however, servlets have no graphical user interface. Servlets can be embedded in many different servers because the servlet API, which you use to write servlets, assumes nothing about the server's environment or protocol. Servlets have become most widely used within HTTP servers; many web servers support the Servlet API.

Use Servlets instead of CGI Scripts!

`SERVLETS` are an effective replacement for CGI scripts. They provide a way to generate dynamic documents that is both easier to write and faster to run. Servlets also address the problem of doing server-side programming with platform-specific APIs: they are developed with the Java Servlet API, a standard Java extension.

So use `SERVLETS` to handle HTTP client requests. For example, have servlets process data posted over HTTPS using an HTML form, including purchase order or credit card data. A servlet like this could be part of an order-entry and processing system, working with product and inventory databases, and perhaps an on-line paymentsystem.

Other Uses for Servlets

Here are a few more of the many applications for servlets:

- Allowing collaboration between people. A servlet can handle multiple requests concurrently, and can synchronize requests. This allows servlets to support systems such as on-line conferencing.
- Forwarding requests. Servlets can forward requests to other servers and servlets. Thus servlets can be used to balance load among several servers that mirror the same content, and to partition a single logical service over several servers, according to task type or organizational boundaries.

Architecture of the Servlet Package

The `javax.servlet` package provides interfaces and classes for writing servlets. The architecture of the package is described below.

The Servlet Interface

The central abstraction in the Servlet API is the `Servlet` interface. All servlets implement this interface, either directly or, more commonly, by extending a class that implements it such as `HTTPServlet`.

The `Servlet` interface declares, but does not implement, methods that manage the servlet and its communications with clients. Servlet writers provide some or all of these methods when developing a servlet.

Client Interaction

When a servlet accepts a call from a client, it receives two objects:

- A `ServletRequest`, which encapsulates the communication from the client to the server.
- A `ServletResponse`, which encapsulates the communication from the servlet back to the client.

The Servlet Request Interface

The `SERVLETREQUEST` interface allows the servlet access to:

- Information such as the names of the parameters passed in by the client, the protocol (scheme) being used by the client, and the names of the remote host that made the request and the server that received it.
- The input stream, `SERVLETInputStream`. `SERVLETs` use the input stream to get data from clients that use application protocols such as the HTTP POST and PUT methods.
- Interfaces that extend `SERVLETREQUEST` interface allow the servlet to retrieve more protocol-specific data. For example, the `HTTPServletRequest` interface contains methods for accessing HTTP-specific header information.

The Servlet Response Interface

The `SERVLETRESPONSE` interface gives the servlet methods for replying to the client.

- It Allows the servlet to set the content length and MIME type of the reply.
- Provides an output stream, `SERVLETOutputStream`, and a `Writer` through which the servlet can send the reply data.
- Interfaces that extend the `SERVLETRESPONSE` interface give the servlet more protocol-specific capabilities. For example, the `HTTPServletResponse` interface contains methods that allow the servlet to manipulate HTTP-specific header information .

Additional Capabilities of HTTP Servlets

The classes and interfaces described above make up a basic Servlet. HTTP servlets have some additional objects that provide session-tracking capabilities. The servlet writer can use these APIs to maintain state between the servlet and the client that persists across multiple connections during some time period. HTTP servlets also have objects that provide cookies. The servlet writer uses the cookie API to save data with the client and to retrieve this data.

The classes mentioned in the Architecture of the Servlet Package section are shown in the example in bold:

- `SIMPLESERVLET` extends the `HTTPSERVLET` class, which implements the `Servlet` interface .
- `Simple SERVLET` overrides the `do Get` method in the `HTTPSERVLET` class. The `doGet` method is called when a client makes a GET request (the default HTTP request method), and results in the simple HTML page being returned to the client.
- Within the `do Get` method, The user's request is represented by an `HTTPSERVLETREQUEST` object.
- The response to the user is represented by an `HTTPSERVLETRESPONSE` object.
- Because text data is returned to the client, the reply is sent using the `Writer` object obtained from the `HTTPSERVLETRESPONSE` object.

Servlets Lifecycle

Each servlet has the same life cycle:

- A server loads and initializes the servlet
- The `SERVLET` handles zero or more client requests
- The server removes the servlet

Initializing a Servlet

When a server loads a servlet, the server runs the servlet's `init` method. Initialization completes before client requests are handled and before the servlet is destroyed.

Even though most servlets are run in multi-threaded servers, servlets have no concurrency issues during servlet initialization. The server calls the `init` method once, when the server loads the servlet, and will not call the `init` method again unless the server is reloading the servlet.

The server can not reload a servlet until after the server has destroyed the `SERVLET` by calling the `destroy` method.

The `init` Method

- The `init` method provided by the `HttpServlet` class initializes the servlet and logs the initialization. To do initialization specific to your servlet, override the `init()` method following these rules:
- If an initialization error occurs that renders the servlet incapable of handling client requests, throw an `UnavailableException`.
- An example of this type of error is the inability to establish a required network connection.
- Do not call the `System.exit` method

Initialization Parameters

- The second version of the `init` method calls the `GETINITPARAMETER` method. This method takes the parameter name as an argument and returns a `String` representation of the parameter's value.
- The specification of initialization parameters is server-specific. In the Java Web Server, the parameters are specified with a servlet is added then configured in the Administration Tool. For an explanation of the Administration screen where this setup is performed, see the Administration Tool: Adding Servlets online help document. If, for some reason, you need to get the parameter names, use the `getparameternames` method.

Destroying a Servlet

Servlets run until the server destroys them, for example at the request of a system administrator. When a server destroys a servlet, the server runs the servlet's `destroy` method. The method is run once; the server will not run that servlet again until after the server reloads and reinitializes the servlet. When the `destroy` method runs, another thread might be running a service request. The Handling Service Threads at Servlet Termination section shows you how to provide a clean shutdown when there could be long-running threads still running service requests.

Using the Destroy Method

The `destroy` method provided by the `HttpServlet` class destroys the servlet and logs the destruction.

To destroy any resources specific to your servlet, override the `destroy` method. The `destroy` method should undo any initialization work and synchronize persistent state with the current in-memory state.

The following example shows the `destroy` method that accompanies the [init](#) method shown previously:

```
Public class BOOKDBSERVLET extends GENERICSERVLET
```

```
{Private BOOKSTORE DB books;
```

```
... // the init method
```

```
    Public void destroy() {
```

```
        // Allow the database to be garbage collected
```

```
        books = null;
```

```
    }
```

```
}
```

A server calls the `destroy` method after all service calls have been completed, or a server-specific number of seconds have passed, whichever comes first. If your servlet handles any long-running operations, service methods might still be running when the server calls the `destroy` method. You are responsible for making sure those threads complete. The next section shows you how the `destroy` method shown above expects all client interactions to be completed when the `destroy` method is called, because the servlet has no long-running operations.

Handling Service Threads at Servlet Termination

All of a SERVLET'S service methods should be complete when a servlet is removed. The server tries to ensure this by calling the `destroy` method only after all service requests have returned, or after a server-specific grace period, whichever comes first. If your servlet has operations that take a long time to run (that is, operations that may run longer than the server's grace period), the operations could still be running when `destroy` is called. You must make sure that any threads still handling client requests complete; the remainder of this section describes a technique for doing this.

If your servlet has potentially long-running service requests, use the following techniques to:

- Keep track of how many threads are currently running the `service` method.
- Provide a clean shutdown by having the `destroy` method notify long-running threads of the shutdown and wait for them to complete
- Have the long-running methods poll periodically to check for shutdown and, if necessary, stop working, clean up and return.

Tracking Service Requests

To track service requests, include a field in your servlet class that counts the number of service methods that are running. The field should have access methods to increment, decrement, and return its value.

The `service` method should increment the service counter each time the method is entered and decrement the counter each time the method returns. This is one of the few times that your `HttpServletRequest` subclass should override the `service` method. The new method should call `super.service` to preserve all the original `HttpServletRequest.service` method's functionality.

Providing a Clean Shutdown

To provide a clean shutdown, your `destroy` method should not destroy any shared resources until all the service requests have completed. One part of doing this is to check the service counter. Another part is to notify the long-running methods that it is time to shut down. For this, another field is required along with the usual access methods.

Servlet-client Interaction

Handling HTTP Clients

An HTTP Servlet handles client requests through its `service` method. The `service` method supports standard HTTP client requests by dispatching each request to a method designed to handle that request. For example, the `service` method calls the `doGet` method shown earlier in the simple example servlet.

Requests and Responses

Methods in the `HTTPServlet` class that handle client requests take two arguments:

- An `HttpServletRequest` object, which encapsulates the data *from* the client
- An `HttpServletResponse` object, which encapsulates the response *to* the client

HTTP Servlet Request Objects

An `HttpServletRequest` object provides access to HTTP header data, such as any cookies found in the request and the HTTP method with which the request was made. The `HttpServletRequest` object also allows you to obtain the arguments that the client sent as part of the request.

To access client data:

The `getParameter` method returns the value of a named parameter. If your parameter could have more than one value, use `getParameterValues` instead.

- The `getParameterValues` method returns an array of values for the named parameter. (The method `getParameterNames` provides the names of the parameters.)
- For HTTP GET requests, the `getQueryString` method returns a `String` of raw data from the client. You must parse this data yourself to obtain the parameters and values.
- For HTTP POST, PUT, and DELETE requests,
- If you expect text data, the `getReader` method returns a `BufferedReader` for you to use to read the raw data.
- If you expect binary data, the `getInputStream` method returns a `ServletInputStream` for you to use to read the raw data

HTTP Servlet Response Objects

An `HTTPServletResponse` object provides two ways of returning data to the user:

- The `getWriter` method returns a `Writer`
- The `getOutputStream` method returns a `ServletOutputStream`

Use the `getWriter` method to return text data to the user, and the `getOutputStream` method for binary data. Closing the `Writer` or `ServletOutputStream` after you send the response allows the server to know when the response is complete.

HTTP Header Data

You must set HTTP header data *before* you access the `Writer` or `OutputStream`. The `HTTPServletResponse` class provides methods to access the header data. For example, the `setContentType` method sets the content type. (This header is often the only one manually set.)

Handling GET and POST Requests

The methods to which the `service` method delegates HTTP requests include,

- `doGet`, for handling GET, conditional GET, and HEAD requests
- `doPost`, for handling POST requests
- `doPut`, for handling PUT requests
- `doDelete`, for handling DELETE requests

By default, these methods return a `BAD_REQUEST` (400) error. Your servlet should override the method or methods designed to handle the HTTP interactions that it supports. This section shows you how to implement methods that handle the most common HTTP requests: GET and POST.

The `HttpServlet`'S `service` method also calls the `doOptions` method when the servlet receives an `OPTIONS` request, and `doTrace` when the servlet receives a `TRACE` request. The default implementation of `doOptions` automatically determines what HTTP options are supported and returns that information. The default implementation of `doTrace` causes a response with a message containing all of the headers sent in the trace request. These methods are not typically overridden.

Servlet Descriptions

In addition to handling HTTP client requests, some applications, such as the Java Web Server's Administration Tool, get descriptive information from the servlet and display it. The servlet description is a string that can describe the purpose of the servlet, its author, its version number, or whatever the servlet author deems important.

The method that returns this information is `GETSERVLETINFO`, which returns `null` by default. You are not required to override this method, but applications are unable to supply a description of your servlet unless you do.

Writing Your First Servlet

Servlets are also easy to develop. This document discusses the following minimum steps needed to create any servlet:

1. Write the servlet
 - a. Import the necessary Java packages
 - b. Inherit from `GENERICServlet` or the HTTP convenience class `HttpServlet`
 - c. Override the `service` method (this is where the actual work is done by the servlet)
 - d. Save the file with a `.java` filename extension
2. Compile the servlet
 - a. Make sure `jws.jar` is included in your class path
 - b. Invoke `javac`
3. Install the servlet
 - a. Use the Java Web Server's Administration Tool to install it, and optionally configure it.
4. Test the servlet
 - a. Invoke the servlet from a JDK1.1-compatible browser.

About Session Tracking

Session T tracking is a flexible, lightweight mechanism that enables STATEFUL programming on the web. Its general implementation serves as a basis for more sophisticated state models, such as persistent user profiles or multi-user sessions.

A session is a series of requests from the same user that occur during a time period. This transaction model for sessions has many benefits over the single-hit model. It can maintain state and user identity across multiple page requests. It can also construct a complex overview of user behavior that goes beyond reporting of user hits.

Server-Side Session Objects and Users

Session tracking gives servlets and other server-side applications the ability to keep state information about a user as the user moves through the site. Server-side applications can use this facility to create more STATEFUL user experiences and to track who's doing what on the site.

Java Web Server maintains user state by creating a Session object for each user on the site. These Session objects are stored and maintained on the server. When a user first makes a request to a site, the user is assigned a new Session object and a unique session ID. The session ID matches the user with the Session object in subsequent requests. The Session object is then passed as part of the request to the servlets that handle the request. Servlets can add information to Session objects or read information from them.

Session Endurance

After the user has been idle for more than a certain period of time (30 minutes by default), the user's session becomes invalid, and the corresponding Session object is destroyed.

A session is a set of requests originating from the same browser, going to the same server, bounded by a period of time. Loosely speaking, a session corresponds to a single *sitting* of a single anonymous user (anonymous because no explicit login or authentication is required to participate in session tracking).

The first part of the `DOGET` method associates the Session object with the user making the request. The second part of the method gets an integer data value from the Session object and increments it. The third part outputs the page, including the current value of the counter.

When run, this servlet should output the value of the counter that increments every time you reload the page. You must obtain the Session object before you actually write any data to the SERVLET'S output stream. This guarantees that the session tracking headers are sent with the response.

The Session object has methods similar to `JAVA.UTIL.DICTIONARY` for adding, retrieving, and removing arbitrary Java objects. In this example, an Integer object is read from the Session object, incremented, then written back to the Session object.

Any name, such as `sessiontest.counter`, may be used to identify values in the Session object. When choosing names, remember that the Session object is shared among any servlets that the user might access. Servlets may access or overwrite each other's values from the Session. Thus, it is good practice to adopt a convention for organizing the namespace to avoid collisions between servlets, such as: `servletname.name`.

Session Invalidation

Sessions can be invalidated automatically or manually. Session objects that haveno page requests for a period of time (30 minutes by default) are automatically invalidated by the Session Tracker `SESSIONINVALIDATIONTIME` parameter. When a session is invalidated, the Session object and its contained data values are removed from the system.

After invalidation, if the user attempts another request, the Session Tracker detects that the user's session was invalidated and creates a new Session object. However, data from the user's previous session will be lost.

Session objects can be invalidated manually by calling `session.invalidate()`. This will cause the session to be invalidated immediately, removing it and its data values from the system.

Handling Non-Cookie Browsers (URL Rewriting)

The Session Tracker uses a session ID to match users with Session objects on the server side. The session ID is a string that is sent as a cookie to the browser when the user first accesses the server. On subsequent requests, the browser sends the session ID back as a cookie, and the server uses this cookie to find the session associated with that request.

There are situations, however, where cookies will not work. Some browsers, for example, do not support cookies. Other browsers allow the user to disable cookie support. In such cases, the Session Tracker must resort to a second method, URL rewriting, to track the user's session.

URL rewriting involves finding all links that will be written back to the browser, and rewriting them to include the session ID. For example, a link that looks like this:

`<a HREF="/store/catalog">` might be

rewritten to look like this:

`<a HREF="/store/catalog;$sessionId$DA32242SSGE2">`

If the user clicks on the link, the rewritten form of the URL will be sent to the server. The server's Session Tracker will be able to recognize the `;$sessionId$DA32242SSGE2` and extract it as the session ID. This is then used to obtain the proper Session object.

Implementing this requires some reworking by the servlet developer. Instead of writing URLs straight to the output stream, the servlet should run the URLs through a special method before sending them to the output stream.

The `ENCODEURL` method performs two functions:

1. **Determine URL Rewriting:** The `ENCODEURL` method determines if the URL needs to be rewritten. Rules for URL rewriting are somewhat complex, but in general if the server detects that the browser supports cookies, then the URL is not rewritten. The server tracks information indicating whether a particular user's browser supports cookies.
2. **Return URL (modified or the same):** If the `encodeurl` method determined that the URL needs to be rewritten, then the session ID is inserted into the URL and returned. Otherwise, the URL is returned unmodified.

In addition to URLs sent to the browser, the servlet must also encode URLs that would be used in `sendRedirect()` calls. For example, a servlet that used to do this:

```
response.sendRedirect("http://myhost/store/catalog");
```

Should now do this:

```
response.sendRedirect(response.encodeRedirectUrl("http://myhost/store/catalog"));
```

The methods `ENCODEURL` and `ENCODEREDIRECTURL` are distinct because they follow different rules for determining if a URL should be rewritten.

Multiple Servlets

URL conversions are required only if the servlet supports session tracking for browsers that do not support cookies or browsers that reject cookies. The consequences of not doing these conversions is that the user's session will be lost if the user's browser does not support cookies and the user clicks on an un-rewritten URL. Note that this can have consequences for other servlets. If one servlet does not follow these conventions, then a user's session could potentially be lost for all servlets.

Using Session Tracking with the Page Compiler

Page compilation is a feature of the Java Web Server that allows HTML pages containing Java code to be compiled and run as servlets. Page compilation also simplifies the task of supporting session tracking. To that end, if URL rewriting is enabled, page compilation automatically adds the `encodeurl` call to links in the HTML page.

Additional APIs

In addition to the Session object, there are a few more classes that may interest the servlet developer.

Description	Class
<code>HttpSessionContext</code>	The <code>HttpSessionContext</code> is the object that contains all existing and valid sessions. The <code>HttpSessionContext</code> can be obtained by calling <code>getSessionContext()</code> on the Session object. The <code>HttpSessionContext</code> lets you find other Session objects by their IDs and list the IDs of all valid sessions.
<code>HttpSessionBindingListener</code>	<code>HttpSessionBindingListener</code> is an interface that can be implemented by objects placed into a Session. When the Session object is invalidated, its contained values are also removed from the system. Some of these values may be active objects that require cleanup operations when their session is invalidated. If a value in a Session object implements <code>HttpSessionBindingListener</code> , then the value is notified when the Session is invalidated, thereby giving the object a chance to perform any necessary cleanup operations.

Session Swapping and Persistence

An Internet site must be prepared to support many valid sessions. A large site, for example, might have hundreds, or even thousands, of simultaneously valid sessions.

Because each session can contain arbitrary data objects placed there by the application servlets, the memory requirements for the entire system can grow prohibitively large.

To alleviate some of these problems, the session tracking system places a limit on the number of Session objects that can exist in memory. This limit is set in the `session.setMaxResidents` property. When the number of simultaneous sessions exceeds this number, the Session Tracker swaps the least recently-used sessions out to files on disk. Those sessions are not lost: they will be reloaded into memory if further requests come in for those sessions. This system allows for more sessions to remain valid than could exist in memory.

Session invalidation is not affected by session swapping. If a session goes unused for longer than the normal invalidation time, the session is invalidated, whether it is in memory or on disk. Session invalidation is set in the `session.invalidateInterval` property.

Sessions are written to and read from disk using Java serialization. For this reason, only serializable objects put into the Session object will be written to disk. Any objects put into the Session object that are not serializable will remain in memory, even if the rest of the Session object has been written to disk. This does not affect session tracking, but does reduce the memory savings that the Session Tracker gets from swapping a session to disk. For this reason, the servlet developer should try to put only serializable objects into the Session object. Serializable objects are those that implement either `java.io.Serializable` or `java.io.Externalizable`.

The session-swapping mechanism is also used to implement session persistence, if the session persistence feature is enabled. When the server is shut down, sessions still in memory are written to the disk as specified in the `session.swapDirectory` property. When the server starts again, sessions that were written to disk will once again become valid. This allows the server to be restarted without losing existing sessions. Only serializable data elements in the session will survive this shutdown/restart operation.

Customizing Session Tracking

Session-tracking interfaces are in the `javax.servlet.http` package.

Properties

You can customize properties in the Session Tracker. The properties are kept in the `server.properties` files

at: `<server_root>/properties/server/javawebserver/server.properties` where `<server_root>` is the directory into which you installed the Java Web Server product.

Note: These property settings are applied to all sessions, and cannot be tuned for individual sessions.

Parameter	Description	Default
<code>session.invalidatationinterval</code>	Time interval when Java Web Server checks for sessions that have gone unused long enough to be invalidated. Value is an integer, specifying the interval in milliseconds.	10000 (10 seconds)
<code>session.swapinterval</code>	Time interval when Java Web Server checks if too many sessions are in memory, causing the overflow of sessions to be swapped to disk. Value is an integer, specifying the interval in milliseconds.	10000 (10 seconds)
<code>session.persistence</code>	Boolean value specifying if Java Web Server keeps session data persistent. If <code>true</code> , sessions are swapped to disk when Java Web Server shuts down and are revalidated from disk when it restarts. If <code>false</code> , Java Web Server removes session swap files every	<code>true</code>

	time it starts.	
<code>session.swapdirectory</code>	Name of directory that the Java Web Server uses to swap out session data. No other data should be kept in this directory.	<code>sessionSwap</code>
<code>session.maxresidents</code>	Number of sessions allowed to remain in memory at once. If the number of sessions exceeds this number, sessions will be swapped out to disk on a least recently used basis to reduce the number of resident sessions.	1024
<code>session.invalidationtime</code>	Amount of time a session is allowed to go unused before it is invalidated. Value is specified in milliseconds.	1800000 (30 minutes)
<code>enable.sessions</code>	Boolean value specifying whether Session Tracking is active. If <code>false</code> , then the Java Web Server performs no function for extracting or inserting session IDs into requests.	<code>true</code>
<code>enable.cookies</code>	Boolean value indicating whether Java Web Server uses cookies as a vehicle for carrying session ID. If <code>true</code> , session IDs arriving as cookies are recognized and the Java Web Server tries to use cookies as a means for sending the session ID.	<code>true</code>
<code>enable.urlrewriting</code>	Boolean value indicating whether Java Web Server uses rewritten URLs as a vehicle to carry the session ID. If <code>true</code> , then session IDs arriving in the URL are recognized, and the Java Web Server rewrites	<code>false</code>

	URLs if necessary to send the session ID.	
<code>enable.protocolswitchrewriting</code>	Boolean value indicating whether the session ID is added to URLs when the URL dictates a switch from "http" to "https" or vice-versa.	false
<code>session.cookie.name</code>	Name of the cookie used to carry the session ID, if cookies are in use.	jwssessionid
<code>session.cookie.comment</code>	Comment of the cookie used to carry the session ID, if cookies are in use.	Java Web Server Session Tracking Cookie
<code>session.cookie.domain</code>	If present, this defines the value of the <code>domain</code> field that is sent for session cookies.	null
<code>session.cookie.maxage</code>	If present, this defines the value of the maximum age of the cookie.	-1
<code>session.cookie.path</code>	If present, this defines the value of the <code>path</code> field that will be sent for session cookies.	"/"
<code>session.cookie.secure</code>	If <code>true</code> , then session cookies will include the <code>secure</code> field.	false

Introduction to MySQL

MySQL is a widely used open-source relational database management system (RDBMS). It is designed to efficiently store, manage, and retrieve data in a structured manner. As an RDBMS, MySQL organizes data into tables with predefined schemas, allowing users to interact with the data using SQL (Structured Query Language).

MySQL is known for its ease of use, performance, and scalability. It is commonly used in web applications and is the default database system for popular content management systems like WordPress. MySQL is compatible with various operating systems and programming languages, making it versatile and widely accessible.

Distinct Features Of MySQL

- **Open-Source:** MySQL is an open-source database, which means it is freely available for use, modification, and distribution. This makes it a cost-effective option for individuals and organizations.
- **Cross-Platform Compatibility:** MySQL is compatible with various operating systems such as Windows, Linux, macOS, and more. It can run on different hardware platforms, making it versatile and widely accessible.
- **Performance and Scalability:** MySQL is known for its excellent performance and scalability. It can handle high volumes of data and concurrent connections efficiently. MySQL provides various optimization techniques, indexing mechanisms, and caching mechanisms to enhance performance.
- **Replication and High Availability:** MySQL offers robust replication features, allowing data to be replicated across multiple servers. This enables redundancy, load balancing, and improved availability. It also supports master-slave replication and master-master replication configurations.
- **Security:** MySQL provides features to ensure the security of data. It supports user authentication, encrypted connections, and access control through user privileges. It also offers features like SSL/TLS encryption for securing data transmission.
- **Full-Text Search:** MySQL includes a powerful full-text search capability, allowing users to perform advanced text searches on indexed text fields. This feature is particularly useful for applications that require searching through large amounts of textual data.
- **Data Integrity and ACID Compliance:** MySQL ensures data integrity by supporting transactional capabilities and adhering to ACID (Atomicity, Consistency, Isolation, Durability) properties. It allows developers to perform operations in a reliable and consistent manner.
- **Community and Support:** MySQL has a large and active community of users and developers. This vibrant community provides extensive documentation, forums, and online resources for support, troubleshooting, and learning.

Chapter 5 : Implementation and Testing

Testing plays a critical role for quality assurance and for ensuring the reliability of the software. Its basic function is to detect the errors. After the coding phase, testing is done to test the proper working of the new system. Testing is the process of executing a program with the intention of finding errors. It is a complete verification to determine whether the objectives are met and the user requirements are satisfied. The testing phase involves testing of a system using various test data.

Preparation of the test data plays a vital role in the system testing. After preparing the test data, the system under study is testing using those test data. Errors were found and corrected by using the following testing steps and corrections are recorded for future references. Thus, a series of testing is performed on the system before it is ready for coding. Since code is the only product that can be executed frequently whose actual behavior can be observed, this phase is so important for the successful implementation of the software product. Thus, the goal of testing is to uncover the requirements, design and coding errors in the program.

Unit Testing

The first step in the testing is the unit testing. Unit test is normally considered as an adjunct to the coding step. After the coding has been developed, received and verified for correct syntax, unit testing begins. The standalone modules were tested individually for their correct functionality, with the corresponding data. This ensures the reliability of the modules when integrated. Each and every module is tested independently with sample data and it was found that all modules are properly functioning. Using the unit test plans, prepared in the design phase of the system as a guide, important control paths are tested to uncover errors within the boundary of the modules. Boundary conditions were checked, all independent paths were exercised to ensure that all statements in the module are checked at least once and all error handling paths were tested. Each unit was thoroughly tested to check if it might fall in any possible situation. This testing was carried out during the programming itself. At the end of this testing phase, each unit was found to be working satisfactory, as regard to the expected output from the module.

Integration Testing

The second step in the testing process is the Integration testing. Integration testing is the systematic technique for constructing the program structure while conducting tests to uncover errors associated with interfacing. All the modules when unit testing will work properly but after interfacing the data can be lost across an interface, one module can have an inadvertent, adverse effect on other, sub functions when combined may not produce the desired major function, global data structures can cause problems, etc.

Integration testing was performed by integrating all the individual modules and the activities of the user such as loading layers, retrieving information from any functions applying themes based on the records present in the database etc. and is found that it works good to the examination of the end users. Hence, the objective of integration testing is to take unit tested modules and build a final program structure.

All the modules developed are independent. Even the whole process of approval for all. Each module is integrated well with other modules. And all the interfaces are tested successfully.

Functional Testing

This test involves testing the system under typical operating conditions with sample input values. Functional testing was performed on the system by giving existing industry id or plot number and a null or string as the input for any field in which case the user should be redirected to the same state with the appropriate message, rather than proceeding and crashing in the system.

Functional testing was performed on the system by raising the demand with an eye to check all the validations. The total processing of the system is satisfactory with the following results.

- All the validations are clearly notified to the user regarding jobseekers reg, new client reg, job order, job providers, and job search preparation etc.
- Almost all the functional errors, data storage errors and all types of logical errors are tested successfully.

Acceptance Testing

User acceptance test of a system is the factor for the success of the system. The system under consideration was listed for user acceptance by keeping constant touch with the perspective user of the system at the time of design, development and making changes whenever required for unit testing.

The requirements of the customer are gathered at regular intervals at the developing site itself. The problems that are to be visualized through this tool are been gathered by the customer and are reported.

The user at the user's site carried this test. Live data entered and the system's output was compared with what was manually prepared. Here the system has met the user's requirement in the following fields:

1. Data Entry
2. Error Handling
3. Reporting and corrections
4. Data Access Protections
5. System Output

Implementation

Implementation includes all those activities that take place to convert the old system to the new system. The new system will replace the existing system. The aspects of implementation are as follows. Conversion, Post Implementation Review.

Conversion

Conversion means changing from one system to another. The objective is to put the tested system into operation. It involves proper installation of the software package developed and training the operating staff.

The software has been installed and found to be functioning properly. The users how to be trained to handle the system effectively. Sample data provide to the operating staff and were asked to operate on the system. The operating staffs now have a clear out look of the software and are ready for practical implementation of the package.

Post Implementation Review

A post implantation review is an evaluation of system in terms of the extent to which the system accomplishes the started objectives. This starts after the system is implemented and conversation is complete.

5.1 User Interface and snapshots

Home Page

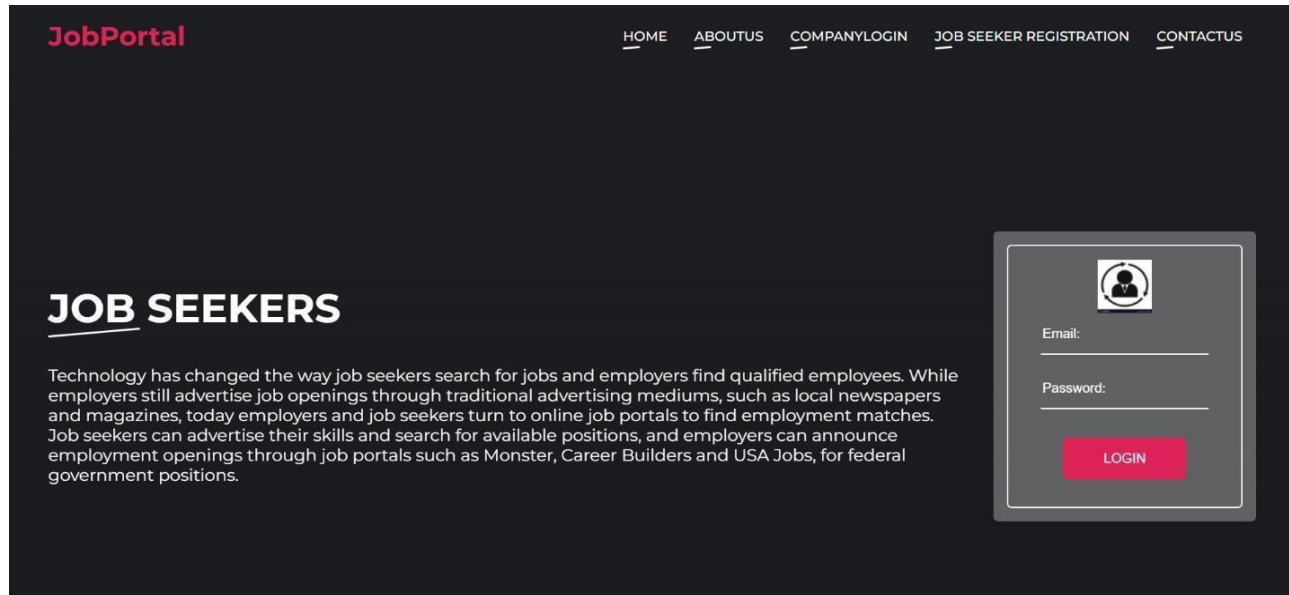


Figure 5.1- Home page I

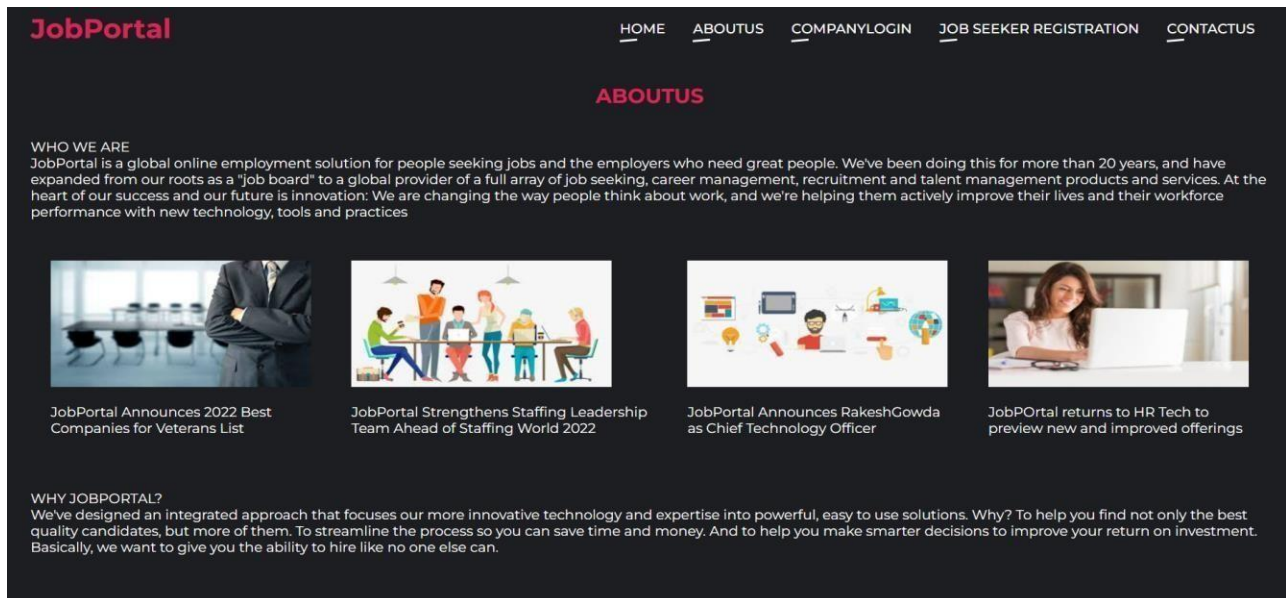
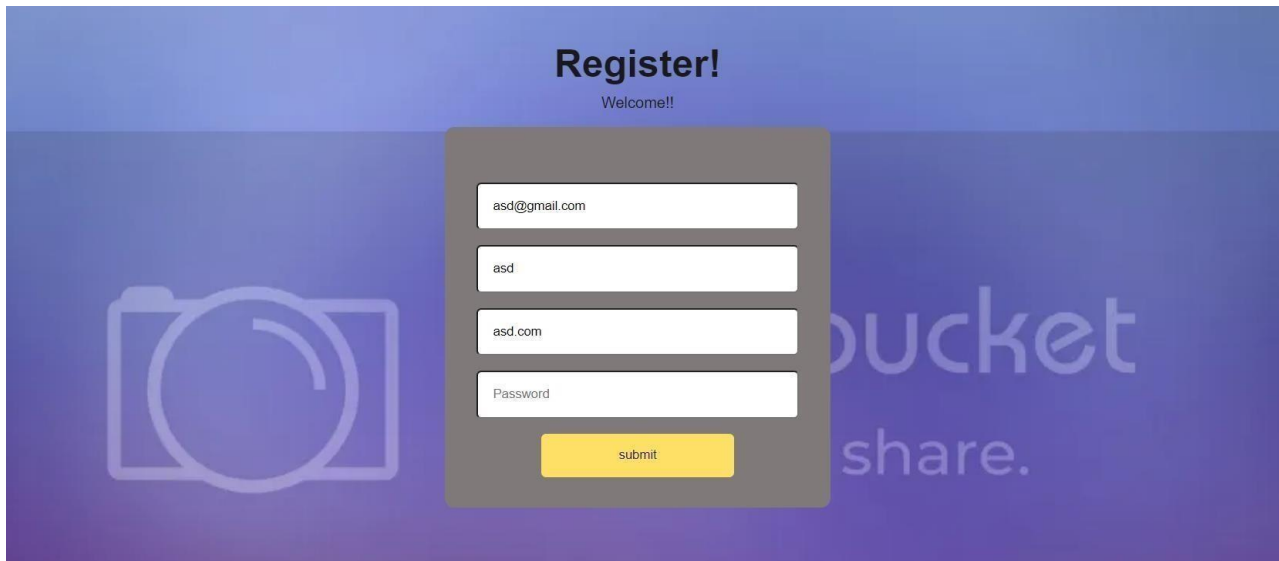


Figure 5.2- Home page II

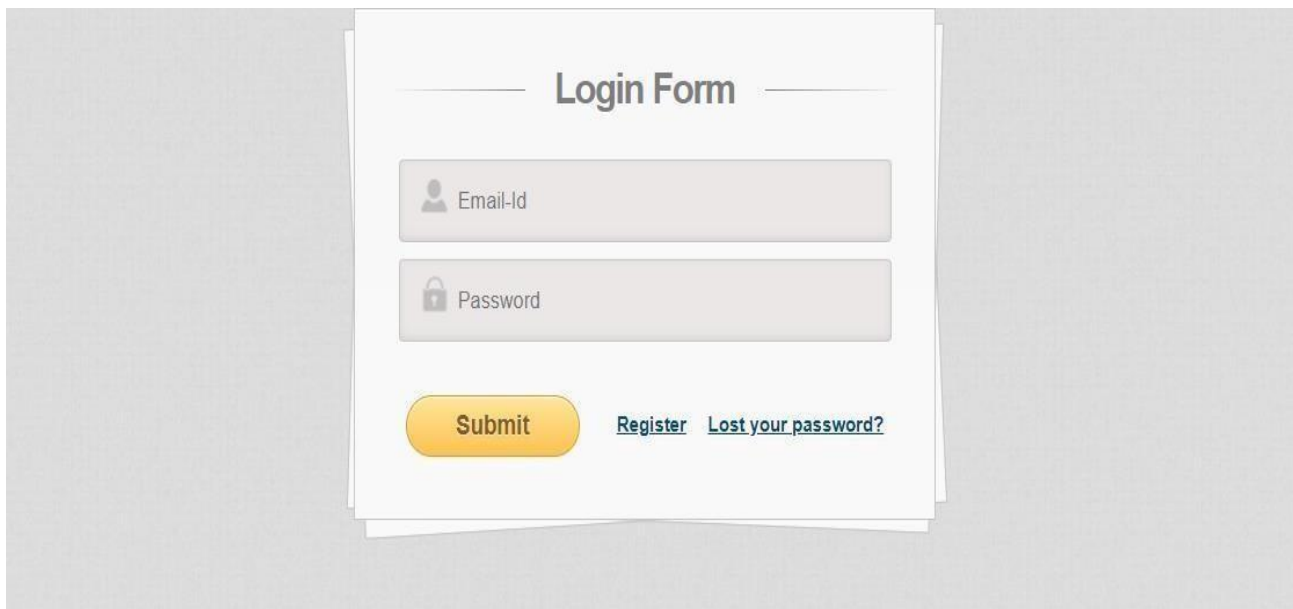
Company Registration Page



The image shows a registration form titled "Register!" with a subtitle "Welcome!!". The form is set against a blue background with a camera icon on the left and the text "bucket share." on the right. The form itself is a dark grey box containing four input fields: "Email" (with "asd@gmail.com" entered), "Username" (with "asd" entered), "Domain" (with "asd.com" entered), and "Password". A yellow "submit" button is at the bottom of the form.

Figure 5.3- Company Registration Page

Company Login Form



The image shows a login form titled "Login Form" on a light grey background. The form is a white box with a shadow, containing two input fields: "Email-Id" (with a person icon) and "Password" (with a lock icon). Below the fields is a yellow "Submit" button. To the right of the button are two links: "Register" and "Lost your password?".

Figure 5.4- Company Login Form

Company Job Posting

FILL UP THE FORM TO POST JOB

Location

FunctionalArea

Job Post

Phone No.:

Vacancy:

Skills Required

Salary

>

Interview Date

Interview Time

Interview Place

Figure 5.5- Company Job Posting

User Registration Form



Sign Up

Please fill in this form to create an account.

Email

Enter Email

First name

Enter First Name

Last name:

Enter Last Name

Desired UserName:

Enter UserName

Password

Enter Password

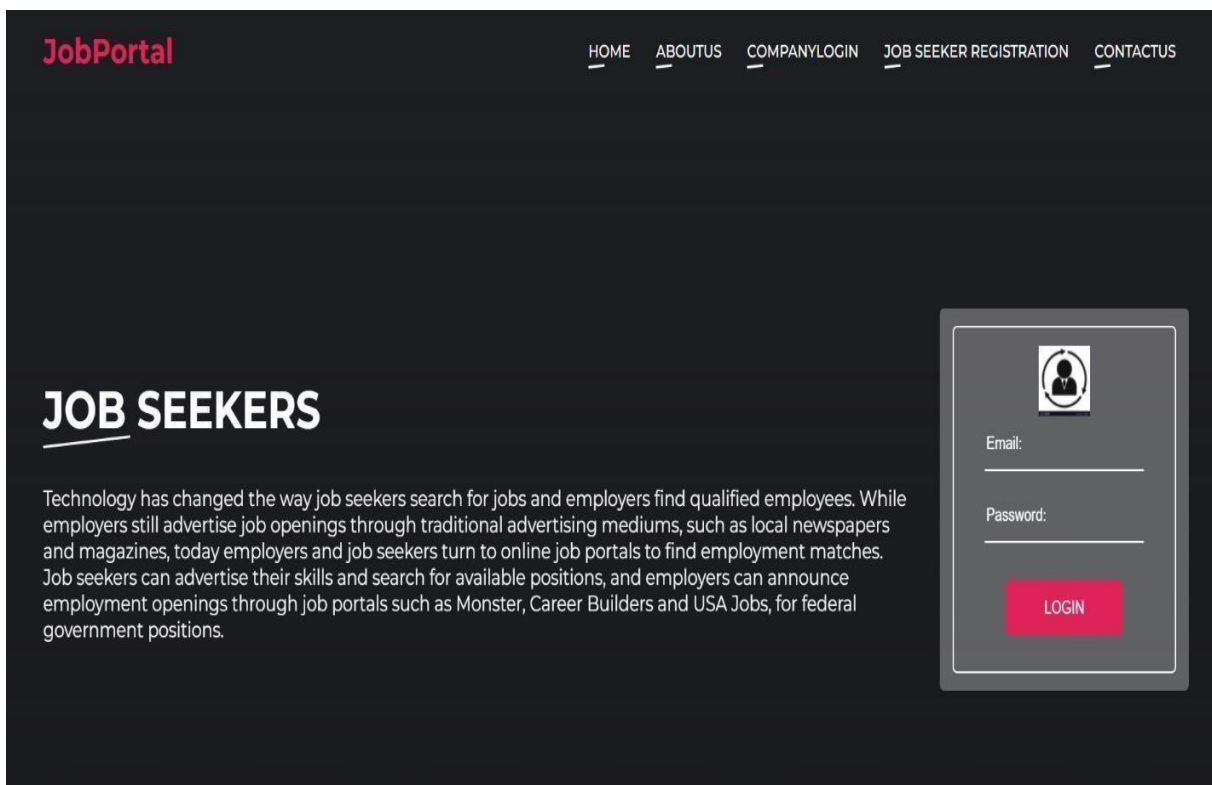
Repeat Password

Repeat Password

Gender ☐ Male ☐ Female

Figure 5.6- User Registration Form

User Login Form



JobPortal

[HOME](#) [ABOUTUS](#) [COMPANYLOGIN](#) [JOB SEEKER REGISTRATION](#) [CONTACTUS](#)

JOB SEEKERS

Technology has changed the way job seekers search for jobs and employers find qualified employees. While employers still advertise job openings through traditional advertising mediums, such as local newspapers and magazines, today employers and job seekers turn to online job portals to find employment matches. Job seekers can advertise their skills and search for available positions, and employers can announce employment openings through job portals such as Monster, Career Builders and USA Jobs, for federal government positions.



Email: _____

Password: _____

LOGIN

Figure 5.7- User Login Form

5.1 Test Cases and Result

Table 12- Test Case

Test Case Id	Test scenario	Test steps	Test data	Expected result	Actual result
UU01	Create employer registration page	1. Open website and clicked on employer registration	Information of employer data to be added	Data is successfully added in employer table	As expected
UU02	Create employee registration page	1. Open website and clicked on employee registration	Information of employee data to be added	Data is successfully added in employee table	As expected
UU03	Crate login page for employer	1. open website and clicked on employer radio button 2. enter username and password	Employer enter valid data	Data is fetch from employer table	As expected
UU04	Crate login page for employee	1. open website and clicked on employee radio button 2. enter username	Employer enter valid data	Data is fetch from employee table	As expected

		and password			
UU05	Post Job page for employer	Employer login successfully then they post your job	Employer filed necessary data to require for recruit employee	Data is added successfully in Job table And also show home page in trending job	As expected
UU06	My job page for employer	Employer see the who is apply for posted job and also delete job and view applicant and active or inactive job them self	Employer see the applicants, delete job, active or inactive job	Successfully delete job and active or inactive job, and also full details of applicants.	As expected
UU07	View applicant page for employer	Employer see applicant name, education, email,id, Phone no and also download resume and	Employee download the resume, send message, approved applicant	Successfully download the resume, send message to employee, and approved or not approved.	As expected

		send message to employee for further query and approved applicant them self			
UU08	Create my account page for employer	1. click on change password then they can change password 2. click on delete account then they can delete account from portal	User change password then directly put new password. and delete account then first asked for security question and answer and it's verify then delete the account	Successfully change password and delete account	As expected
UU09	Add skills page for employee	Add skills, interested category, and interested city	Employee add their skills, selected city, interested category and also skip this	Successfully data added employee table	As expected

UU10	Wishlist page for employee	1.click on more details page of jobs then applied for add to wishlist	Employee see the employer name, category, location, salary, remove from wish list	Add into wishlist page successfully	As expected
UU11	My jobs page for employee	1.click on more details page of jobs then applied for add to my jobs	Employee see employer name, category , location, add to link for more details, check status of application it is approved or not and remove from my jobs	Add into my jobs page successfully	As expected
UU12	Create my account page for employer	1. click on change password then they can change password 2. click on delete account then	User change password then directly put new password. And delete account then first asked for security	Successfully change password and delete account	As expected

		they can delete account fromportal	question and answer and it's verify then delete the account		
UU13	Message page for employee	Which employer send message employee can see	Employee see employer name, subject, Description, date	Successfully data fetch from message table	As expected
UU14	About us page	1.click on about us page	See the details of our	See the details of our	As expected
UU15	Contact page	1.click on contact page	User can any query then send message us	See the message of sender from admin side	As expected

Chapter 6 : Conclusion and Future Scope

Conclusion

It has been a great pleasure for me to work on this exciting and challenging project. This project proved good for me as it provided practical knowledge of not only programming in ASP.NET and VB.NET web based application and no some extent Windows Application and SQL Server, but also about all handling procedure related with project . It also provides knowledge about the latest technology used in developing web enabled application and client server technology that will be great demand in future. This will provide better opportunities and guidance in future in developing projects independently.

Benefits:

The project is identified by the merits of the system offered to the user. The merits of this project are as follows: -

- It's a web-enabled project.
- This project offers user to enter the data through simple and interactive forms. This is very helpful for the client to enter the desired information through so much simplicity.
- The user is mainly more concerned about the validity of the data, whatever he is entering. There are checks on every stages of any new creation, data entry or updation so that the user cannot enter the invalid data, which can create problems at later date.
- Sometimes the user finds in the later stages of using project that he needs to update some of the information that he entered earlier. There are options for him by which he can update the records. Moreover there is restriction for his that he cannot change the primary data field. This keeps the validity of the data to longer extent.
- User is provided the option of monitoring the records he entered earlier. He can see the desired records with the variety of options provided by him.
- From every part of the project the user is provided with the links through framing so that he can go from one option of the project to other as per the requirement. This is bound to be simple and very friendly as per the user is concerned. That is, we can sat that the project is user friendly which is one of the primary concerns of any good project.

- Data storage and retrieval will become faster and easier to maintain because data is stored in a systematic manner and in a single database.
- Decision making process would be greatly enhanced because of faster processing of information since data collection from information available on computer takes much less time than manual system.
- Allocating of sample results becomes much faster because at a time the user can see the records of last years.
- Easier and faster data transfer through latest technology associated with the computer and communication.
- Through these features it will increase the efficiency, accuracy and transparency .

Future Work

- This System being web-based and an undertaking of Cyber Security Division, needs to be thoroughly tested to find out any security gaps.
- A console for the data centre may be made available to allow the personnel to monitor on the sites which were cleared for hosting during a particular period.
- Moreover, it is just a beginning; further the system may be utilized in various other types of auditing operation viz. Network auditing or similar process/workflow based applications.
- . It also provides knowledge about the latest technology used in developing web enabled application and client server technology that will be great demand in future. This will provide better opportunities and guidance in future in developing projects independently.

References

Books

- Conallen, J. (2003). Building Web Applications with UML. Pearson Education, Inc.
- Firth, R. (2005). High Powered CVs. How to Books Publishing.
- Maier, M. & Rechtin, E. (2000). The Art of Systems Architecting 2nd Edition. CRC Press.
- Schmuller, J. (2004). Sams Teach Yourself UML in 24 Hours, Third Edition. Sams Publishing.

Articles/Journals

- Boyce, A. (2002). Online Job Hunting: A Pew Internet Project Data Memo. Pew Internet and
 - American Life Project. Published: July 17, 2002.
- Dewar, J. (2008). Using an Employment Agency to Best Effect. Success @ Work. Published:
 - January 2008.
- Dorn, J. & Naz, T. (2007). Meta-Search in Human Resource Management. International
 - Journal of Social Sciences.
- Kuhn, P. & Skuterud, M. (2000). Job Search Methods:
 - Internet versus Traditional. Monthly Labor Review. Published: October 2000.

Websites

- <http://www.bestjobs.ph>
- <http://www.gopinoy.com>
- <http://www.jobsdb.com.ph>
- <http://www.jobsonline.com.ph>
- <http://www.jobstreet.com.ph>
- <http://www.phil-job.net>
- <https://www.freshersworld.com>